



universität
ulm

Universität Ulm
Technische Hochschule Ulm

Bericht fortgeschrittenes Projekt in CSE

Umsetzung von SLAM-basierter Onlinekartierung und autonomer Trajektorienabfahrt in Software und Hardware am Testfahrzeug Nimbulus-E

im Sommersemester 2024

vorgelegt von

Matthias Schmitz
1159244
matthias.schmitz@uni-ulm.de

Leonie Seika
1047861
leonie.seika@uni-ulm.de

Theo Nonnenmacher
1014114
theo-1.nonnenmacher@uni-ulmd.de

Erstprüfer: Prof. Dr. Ing. Marcel Mayer

Zweitprüfer: Prof. Dr. Ing. Thomas Schmitz

Abgabedatum: TODO

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Projektübersicht . . . 1
1.2 Hintergrund . . . 1
1.3 Ziele . . . 2
1.3.1 Weitere Gründe für Änderungen und Übernahmen von Aufgaben . . . 3

2 Hardware **4**
2.1 Fahrzeugrohkarosserie . . . 4
2.2 Motoren und Controller . . . 5
2.2.1 Schrittmotor und Controller . . . 5
2.2.2 Radnabenmotoren . . . 6
2.2.3 Verdrahtung und Verkabelung . . . 7
2.3 Sensoren . . . 10
2.3.1 Absolutwertgeber . . . 10
2.3.2 Ultraschallsensoren . . . 11
2.3.3 Kompass . . . 11
2.3.4 Lidar . . . 12
2.4 Sicherheitsfunktionen . . . 12

3 Software **14**
3.1 Xbox-Controller Fahrzeugsteuerung . . . 14
3.1.1 Kommunikation während Xbox-Controller Steuerung . . . 15
3.1.2 Steuerung mit dem Xbox-Controller . . . 17
3.1.3 Inverse Kinematik bei Xbox-Controller Steuerung . . . 19
3.1.4 SLAM-Algorithmus . . . 22
3.2 Autonome Fahrzeugsteuerung . . . 23
3.2.1 Kommunikation während autonomer Fahrzeugsteuerung 24
3.2.2 Berechnung der Testtrajektorien . . . 25
3.2.3 Berechnung der Trajektorie . . . 26
3.2.4 Umrechnung in Lenkwinkel und Drehzahlen . . . 28
3.2.5 Inverse Kinematik bei autonomer Fahrt . . . 33
3.3 Kalibrierung der Absolutwertgeber . . . 33

4 Endergebnisse **35**
4.1 Xbox-Controller Steuerung und SLAM-Ergebnisse 35
4.2 Trajektorienplanung und Ausführung 37

5 Ausblick **39**

6 Fazit **40**

Literatur **41**

A Anleitungen **i**
A.1 Benötigte Toolboxen i
A.2 Hardware-Setup i
A.3 Software-Setup Xbox-Controller Steuerung ii
A.4 Software-Setup autonome Fahrt iv
A.5 Lidar nutzen iv
A.6 One Wheel Steering v
A.7 Vesc Bibliothek in Arduino vi
A.8 Xbox-Controller Steuerung Inbetriebnahme vi
A.9 Autonomes Fahren vii

B Code-Listings **viii**

1 Einleitung

In der heutigen Zeit ist das Thema autonome Fahrzeuge in aller Munde. Sie versprechen eine Zukunft, in der Verkehrssicherheit, Effizienz und Komfort neue Höhen erreichen. Doch der Einsatz von autonomen Fahrzeugen ist nicht nur in der Personenbeförderung von Bedeutung. Auch in Bereichen wie der Landwirtschaft oder der Logistik gewinnen selbstfahrende Plattformen immer mehr an Bedeutung. In diesem Kontext wurde im Rahmen dieses Projekts ein autonomes Fahrzeug mit acht Freiheitsgraden (FHG) weiterentwickelt. Jedes der vier Räder ist einzeln in Drehzahl und Lenkwinkel steuerbar. Zusätzlich ist das Fahrzeug mit weiterer Sensorik wie u. a. einem Kompass und einem Lidar-Sensor ausgestattet, der eine detaillierte 2D-Karte der Umgebung erstellt. Auf der Grundlage dieser Karte plant das Fahrzeug eine Trajektorie und ist in der Lage, der berechneten Trajektorie autonom zu folgen.

Dieser Bericht dokumentiert die Arbeit, die während des Projekts geleistet wurde, und präsentiert die erreichten Ergebnisse.

1.1 Projektübersicht

Das Hauptziel des Projekts war die Entwicklung eines autonomen Fahrzeugs, das in der Lage ist, seine Umgebung zu kartieren, eine Trajektorie zu planen und dieser zu folgen. Um dieses Ziel zu erreichen, wurden verschiedene Hardwarekomponenten und Softwarealgorithmen implementiert und getestet. Diese umfassen unter anderem den Lidar-Sensor zur Kartenerstellung, einen Xbox-Controller zur manuellen Steuerung des Fahrzeugs, verschiedene Motoren und Controller zur Bewegung des Fahrzeugs, sowie verschiedene Algorithmen zur Trajektorienplanung, -verfolgung und -aufzeichnung.

1.2 Hintergrund

Die ursprüngliche Projektidee - schon vor dem Einstieg unserer Gruppe - war es, eine autonome Transportplattform im Maßstab 1:3 zu designen. Diese Plattform sollte als Prototypenmodell in engen Umgebungen in der Lage sein, besondere Lenkmanöver auszuführen wie in besonders schmalen Gassen

durch kombinierte Vorder- und Hinterradlenkung enge Kurven zu fahren. Eine geringe Bodenfreiheit sowie eine niedrige Ladefläche sollten das Beladen der Plattform vereinfachen.

Das Projekt baut auf der Arbeit vorheriger Gruppen auf, die bereits Grundlagen für den Aufbau des Fahrzeug geschaffen haben. Diese umfassten unter anderem die mechanische Konstruktion des Fahrzeuggestells samt Radaufhängung und Lenksystem sowie die grundlegende Stromversorgung aller elektronischen Komponenten.

Ursprünglich sollten durch vorhergehende Arbeiten ebenfalls die Ansteuerung der Radnabenmotoren sowie der Schrittmotoren für die Lenkung implementiert sein. Auch die Installation von Endanschlägen und die softwareseitige Einbindung waren Aufgaben anderer Projektgruppen. Leider wurden diese Ziele zuvor nicht vollständig erreicht und mussten von uns übernommen werden.

Eine stichpunktartige Liste über die vorhergehenden Aufgaben ist zur Übersichtlichkeit erstellt worden:

- **Fahrzeuggestell:** Konstruktion und Zusammenbau des Fahrzeuggestells mittels Vierkant-Rechteck-Aluminiumprofilen
- **Elektrische Komponenten:** Auswahl der elektrischen Komponenten wie Radnabenmotoren, Radnabenmotorcontroller, Schrittmotoren, Schrittmotorcontroller, Stromversorgung über 12 V Fahrradakkus
- **Elektrische Verschaltung:** Grundlegende Verdrahtung der elektrischen Komponenten in einer Hauptsteuerbox sowie in vier kleinen Steuerboxen - jeweils eine pro Rad
- **Radaufhängung und Lenksystem:** Konstruktion der Radaufhängung sowie des Lenksystems samt eines selbst hemmenden Schneckengetriebes zur Ansteuerung des Lenkwinkels. Konstruktion und 3D-Druck der Gehäuse des Lenksystems

1.3 Ziele

Die ursprünglichen Ziele des Projekts umfassten die Aufnahme der 2D-Karte mit dem Lidar-Sensor, die Trajektorienplanung, die Bestimmung der acht

Steuergrößen zur Trajektorienabfahrt sowie die Regelung der abgefahrenen Trajektorie. Eine Regelung sollte nicht nur für das versehentliche Verlassen der ursprünglichen Trajektorie, sondern auch für das plötzliche Auftauchen zuvor nicht kartierter Hindernisse auf der Trajektorie implementiert werden.

Allerdings waren einige der vorhergehenden Arbeiten unvollständig, was zu zusätzlichen Herausforderungen während des Projekts führte. So mussten beispielsweise die Ansteuerung der Controller und die Verdrahtung der Motoren überarbeitet und verbessert, teilweise sogar neu konzipiert werden. Zusätzlich wurden mechanische Konstruktionen an Radaufhängung und Lenksystem verändert, Teile neu gefertigt sowie das Fahrzeug an betroffenen Stellen umgebaut. Darüber hinaus mussten verschiedene Sicherheitsfunktionen implementiert werden, darunter Notstoppfunktionen und Endanschläge.

Trotz dieser zusätzlichen Herausforderungen konnte ein autonomes Fahrzeug entwickelt werden, welches in der Lage ist, seine Umgebung zu kartieren und einer geplanten Trajektorie zu folgen.

1.3.1 Weitere Gründe für Änderungen und Übernahmen von Aufgaben

Die Notwendigkeit, zusätzliche Aufgaben zu übernehmen, ergab sich außerdem aus unvorhergesehenen Hardwareproblemen. Dazu gehörten u. a. nicht zuverlässig zählende Schrittmotoren oder defekte Hallsensoren in den Radnabenmotoren.

Insbesondere waren die Ansteuerung von Radnaben- und Schrittmotorcontrollern sowie die Verdrahtung der Motoren zuvor nicht vollständig ausgearbeitet. Der Einbau von drei verschiedenen Radnabencontrollern mit unterschiedlichen Eigenschaften sowie die Einarbeitung in jeweilige Ansteuerungen/Regelungen führten ebenfalls zu ursprünglich nicht geplanten Arbeitsfeldern.

Diese unvorhergesehenen aber sehr grundlegenden Aufgaben stellten erhebliche Hindernisse für die weitere Entwicklung des Fahrzeugs dar und mussten daher vorrangig behoben werden, bevor die eigentlichen Projektziele bearbeitet werden konnten.

2 Hardware

TODO THEO: Verweise auf Datenblätter als Literatur mit \cite dass Link nicht nur hinter 'hier' versteckt ist?

Die Hardware des Nimbulus-E besteht aus verschiedenen Komponenten, die alle zusammenarbeiten müssen, um die Funktionalität des Fahrzeugs zu ermöglichen. Diese Komponenten umfassen

- die Fahrzeugrohkarosserie
- die Motoren und deren Controller
  - Radnabenmotoren für den Antrieb
  - Schrittmotoren zur Lenkwinkelsteuerung
- die elektronischen Grundbestandteile
  - die beiden Arduinos
  - die Gesamtverkabelung
- die Sensoren
  - Lidar
  - 9-Achsen Orientierungssensor
  - Lenkwinkelsensoren
  - Ultraschallsensoren
- Sicherheitsbauteile

Im Folgenden wird auf alle Komponenten genauer eingegangen.

2.1 Fahrzeugrohkarosserie

Das Rahmenkonzept des Nimbulus-E wurde in der Arbeit von Capellaro [1] entwickelt. Es wurde sich für ein Item-Profilkonzept entschieden. Dafür wurden 25 x 25 x 2 mm Quadratrohre zurecht geschnitten und mit passenden Verbindungsstücken verschraubt. Bei dem Material der Streben handelt es sich um Aluminium 6060, welches aus der Legierung EN AW-AlMgSi mit einer

Dynamischen Streckgrenze von 160 N/mm2 gefertigt ist. Für das Rahmenkonzept wurde speziell darauf geachtet, die gewünschte maximale Belastung von 100 kg zu gewährleistet.

Zudem zählen auch diverse 3D-gedruckten Gehäuse für die Motoren, Controller und Ultraschallsensoren zur Rohkarosserie-Ausstattung des Nimbulus-E. Eine Hartplastikbox am hinteren Teil des Fahrzeugs bildet die zentrale Steuereinheit, in der die gesamte Verkabelung zusammenläuft und die Software adaptiert wird. [2]

2.2 Motoren und Controller

2.2.1 Schrittmotor und Controller

Schrittmotorcontroller:
Bei dem Schrittmotorcontroller handelt es ich um den Closed-Loop CL57T(V4.0) Schrittmotorcontroller (0-8.0A, 24-48VDC) für NEMA 17, 23 und 24 Schrittmotoren. Er arbeitet mit einer Betriebsspannung von 24 bis 48 V und bietet einen einstellbaren Ausgangsstrom von bis zu 8 A, was für anspruchsvolle Anwendungen geeignet ist.

Über eine Pinkonfiguration (vgl. Abb. 44 in [3]) kann die Anzahl der Schrittmotorschritte pro voller Umdrehung des Schrittmotors verändert werden. Aufgrund des vorhandenen Schneckengetriebes zwischen Schrittmotor und Lenkachse mit einer Übersetzung von 18:1 (vgl. [2]) wurde die größtmögliche Schrittweite eingestellt, was 200 Steps des Schrittmotors für eine Umdrehung der Schrittmotorachse entspricht. Mit dieser Übersetzung entsprechen 10 Schritte einem Grad im Lenkwinkel. **TODO THEO: Stellung der Pins wie wir sie am Ende haben für jedes Rad angeben und erwähnen, dass auf allen vier Rädern 'dirPin = HIGH' einer Drehung mathematisch positiv also gegen den Uhrzeigersinn entspricht** Sollte die Schrittgröße des Schrittmotors angepasst werden wollen, müssen zwangsläufig auch Änderungen im Arduino 2 Programmcode vorgenommen werden.

Das Datenblatt kann hier im Download-Bereich der Herstellerwebsite heruntergeladen werden. [4]

Durch unvorhergesehene Ereignisse ist es möglich, dass der Schrittmotorcontroller den Schrittmotor nicht mehr ansteuert. Die Lampen des Controllers leuchten in verschiedenen Kombinationen. Die häufigste Fehlerquelle ist ein für den Schrittmotor zu hohes Gegenmoment, meist durch das Verfangen einer Antriebsleitung ausgelöst. Die rote Lampe leuchtet in diesem Fall 7 Mal kurz auf.

Um diesen Fehler zu beheben, muss zuerst der Wagen ausgeschaltet werden. Der Schrittmotor kann nur ohne Bestromung von außen in seiner Position verändert werden. Es muss per Hand an der Kupplung zwischen Schrittmotor und Getriebe gedreht werden, sodass die eingeklemmte Leitung freigelegt wird und erneut richtig positioniert werden kann. Der Fehlercode sollte nach erneutem Anschalten des Wagens verschwunden sein.

Andere Fehlercodes sind in der Dokumentation des Controllers beschrieben, downloadbar hier. [5]

Schrittmotor:
Der S-Serie NEMA 24 Closed-Loop Schrittmotor ist ein leistungsstarker Schrittmotor, der mit einem integrierten 1000-RPR-Encoder (4000 CPR) ausgestattet ist, um präzise Positionsrückmeldungen und eine effiziente Steuerung zu gewährleisten. Mit einem Drehmoment von 3,0 Nm ist dieser Motor für anspruchsvolle Anwendungen konzipiert, bei denen hohe Genauigkeit und stabile Leistung entscheidend sind. Das technische Datenblatt des Schrittmotors kann hier heruntergeladen werden. [6]

Der Closed-Loop-Betrieb sorgt dafür, dass der Motor auch bei Überlast weiterhin verlässlich Schritte vornimmt und zählt. Es wird eine bessere Energieeffizienz und eine reduzierte Wärmeentwicklung im Vergleich zu herkömmlichen offenen Schrittmotoren erreicht. Weitere technische Details über die Schrittmotoren und deren Controller finden sich in der Bachelorarbeit von Jonathan Tasler [3].

2.2.2 Radnabenmotoren

Bei den Radnabenmotoren handelt es sich um Vbest life 36V 350W Nabenmotoren für 8 Zoll Elektroroller Balance Scooter. Sie haben einen Durchmesser von ca. 20 cm und wiegen 2,4 kg.

Die gesamte Steuerung der Radnabenmotoren erfolgt über eine UART Verbindung zwischen dem Arduino 1 und dem Mastervesccontroller, hinten links (HL), mit der ID 0. Untereinander sind alle VESC-Controller per CAN-Bus (jeweils CAN Low und CAN High) verbunden. Interagiert wird mit den Controllern dann über ihre vordefinierten IDs mit Hilfe der Klasse VescNimbulus.

Die privaten Variablen der Klasse sind: die Anzahl der Polpaare des Motors (hier 15), der Umrechnungsfaktor von elektrischen Umdrehungen pro Minute (ERPM) in Umdrehungen pro Sekunde (RPS) und der Zeiger auf ein VescUart Objekt. In der Initialisierung wird die serielle Kommunikation über Serial 1 mit der definierten Baudrate hergestellt. Zum Ansteuern und Auslesen der jeweiligen Drehzahlen für alle vier Radnabenmotoren wurden die Funktionen `setRPS()` und `getRPS()` von Thomas Zimmermann [7] entwickelt. In dieser Arbeit finden sich auch weitere Details zur Ansteuerung der Radnabenmotoren über die VESC-Controller.

TODO THEO: Wie heißt der Wert in der Vesc konfig und was hat er für nen Wert Angepasst wurde noch die minimale Drehzahl, ab welcher die VESCs anfangen den Motor anzusteuern. Setzt man diesen Wert auf 0, ist der Controller nicht in der Lage den Motor vollkommen ruhig zu halten und bei einem Signal von 0 RPS zittert der Motor hin und her. Allerdings führt ein zu hoher Wert zu signifikanten Fehlern beim autonomen Abfahren einer Trajektorie. Ist die berechnete Drehzahl im nicht ansteuerbaren Bereich entstehen automatisch Fehler.

2.2.3 Verdrahtung und Verkabelung

In der Hartplastikbox (Steuereinheit) befinden sich zwei Arduino Mega. Jedes elektronische Bauteil des Fahrzeugs läuft hier zusammen und wird von der auf den Arduinos integrierten Software gesteuert. Die Pinbelegung der Arduinos kann den Tabellen 1 und 2 entnommen werden.

Die Informationen über die anzusteuernden Größen (Raddrehzahlen & Lenkwinkel) erhält der Arduino 1 über die Verbindung zum Computer (MATLAB) und gibt die gewünschten Lenkwinkel dann mit Hilfe einer weiteren seriellen Verbindung zwischen den Arduinos an den Arduino 2 weiter. Um die Sicherheitsfunktionen, die Auswertung der Kompassdaten, die Drehzahl Ansteue-

rung über die VESC Controller sowie die Auswertung der Absolutwertgeber kümmert sich der Arduino 1 selbst.

TODO THEO: die Serial 0 Schnittstelle wird über USB und nicht über die Pins 0 und 1 genutzt

Pinnumer	Funktion	Kommentar
0	Serial0 RX	Serielle Kommunikation mit dem Computer, nur per USB-Kabel
1	Serial0 TX	Serielle Kommunikation mit dem Computer, nur per USB-Kabel
2	Endanschlag	Interrupt-Eingang für Endanschläge
3	Not-Aus-Schalter	Interrupt-Eingang für Notfallabschaltung
16	Serial2 TX	Serielle Übertragung an Arduino 2
17	Serial2 RX	Serieller Empfang von Arduino 2
18	Serial1 TX	Serielle Übertragung an VESC
19	Serial1 RX	Serieller Empfang vom VESC
20	SDA (I2C)	I2C-Datenleitung für den BNO055-Sensor (Kompass)
21	SCL (I2C)	I2C-Taktleitung für den BNO055-Sensor (Kompass)
48	LED Grün	Grüne Debug-LED
50	LED Gelb	Gelbe Debug-LED
52	LED Rot	Rote Debug-LED
A1	Encoder HL	Analoger Eingang für hinteren linken Absolutwertgeberencoder
A5	Encoder VL	Analoger Eingang für vorderen linken Absolutwertgeberencoder
A9	Encoder HR	Analoger Eingang für hinteren rechten Absolutwertgeberencoder
A13	Encoder VR	Analoger Eingang für vorderen rechten Absolutwertgeberencoder

Tabelle 1: Pinbelegung Arduino 1

Die Stromversorgung erfolgt mit Lithium-Ionen Batterien der Marke X-go mit den Spezifikationen 36 V, 10Ah und 360Wh. Nähere Informationen zur Verschaltung der Akkus, einem 12V auf 230 V Umrichter, dem Einbau des Funk-Not-Aus-Schalters und weiterer Bauteile findet sich in der Arbeit von Louis de Veer. [2]

Pinnumer	Funktion	Kommentar
0	Serial0 RX	Serielle Kommunikation mit dem Computer
1	Serial0 TX	Serielle Kommunikation mit dem Computer
16	Serial2 TX	Serielle Übertragung an Arduino 1
17	Serial2 RX	Serieller Empfang von Arduino 1
34	Richtung VR	Ausgang für Richtungssteuerung des vorderen rechten Rads
36	Richtung HL	Ausgang für Richtungssteuerung des hinteren linken Rads
38	Richtung HR	Ausgang für Richtungssteuerung des hinteren rechten Rads
40	Richtung VL	Ausgang für Richtungssteuerung des vorderen linken Rads
42	Schritt HR	Ausgang für Pulsschritte des hinteren rechten Rads
44	Schritt HL	Ausgang für Pulsschritte des hinteren linken Rads
46	Schritt VL	Ausgang für Pulsschritte des vorderen linken Rads
48	Schritt VR	Ausgang für Pulsschritte des vorderen rechten Rads
A1	Encoder HL	Analoger Eingang für hinteren linken Absolutwertgeberencoder
A5	Encoder VL	Analoger Eingang für vorderen linken Absolutwertgeberencoder
A9	Encoder HR	Analoger Eingang für hinteren rechten Absolutwertgeberencoder
A13	Encoder VR	Analoger Eingang für vorderen rechten Absolutwertgeberencoder

Tabelle 2: Pinbelegung Arduino 2

2.3 Sensoren

2.3.1 Absolutwertgeber

Bei den Absolutwertgebern handelt es sich um auf die Lenkwellen geklebte Magneten, deren Ausrichtung durch einen über ihnen befindlichen Sensor aufgefasst wird. Sie sind ein wichtiger Teil des Lenksystems des Fahrzeugs und ermöglichen eine präzise Messung des Lenkwinkels. Die Verarbeitung der Sensorsignale erfolgt unabhängig auf beiden Arduinos und wurde für beide Microcontroller einzeln kalibriert, da trotz derselben anliegenden Spannung die Analog-to-Digital-Converter (ADC) unterschiedliche Spannungen messen.

Der Grund für die gleichzeitige Signalauswertung auf beiden Aurdinos liegt darin, dass der Arduino 1 die Ist-Werte für die Rückverfolgung der tatsächlichen Trajektorie an MATLAB sendet und der Arudino 2 die Ist-Werte zur genauen Lenkwinkelansteuerung benötigt.

Die Pinbelegung der Absolutwertgeber auf den Arduinos kann den Tabellen 1 und 2 entnommen werden.

Die Rückrechnung des vom Absolutwertgeber zurückgegebenen analogen Spannungswerts erfolgt über eine Mappingfunktion. Zur Aufstellung der Mappingfunktion wurden für jedes Rad die Winkel $+90°$ und $-90°$ angefahren, sowie die anliegenden Spannungen gemessen. So kann über einen linearen Zusammenhang der anliegende Spannungswert in den angefahrenen Winkel umgerechnet werden.

Die gemessenen Spannungen zwischen ideal 1,25V und 3,75V bei $+90°$ und $-90°$, die per 10 Bit ADC als digitale Werte zwischen ideal 256 und 768 Bit dargestellt werden, können Tabelle 3 entnommen werden.

	Bits bei $-90°$		Bits bei $0°$		Bits bei $+90°$	
Rad	Ardu 1	Ardu 2	Ardu 1	Ardu 2	Ardu 1	Ardu 2
VL	240	252	514	552	791	847
VR	237	247	511	550	779	836
HL	224	233	503	540	772	829
HR	224	233	500	537	772	828

Tabelle 3: Offsets der Absolutwertgeber

2.3.2 Ultraschallsensoren

TODO THEO: Qualität der Grafik verbessern
TODO MATTHES: Grafik weglassen

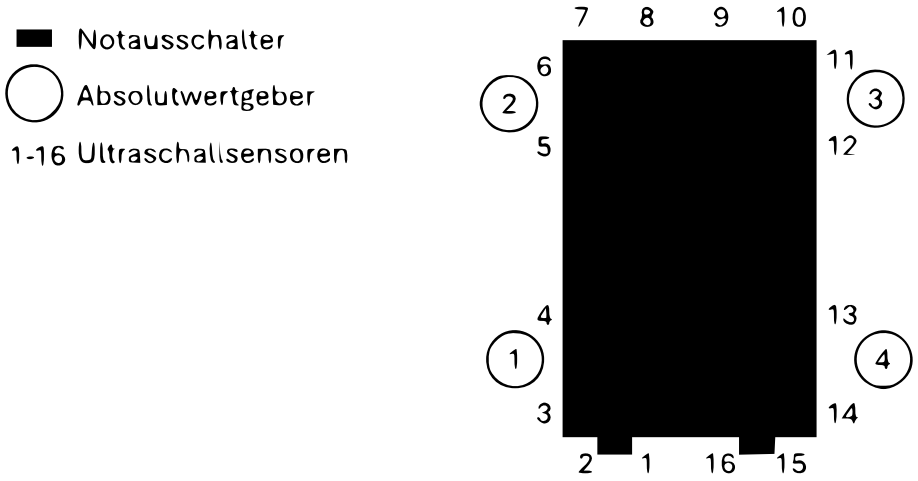


Abbildung 1: Schematische Skizze Ultraschallsensoren

Am gesamten Fahrzeug sind insgesamt 16 HCSR04-Ultraschallsensoren platziert. Sie gewährleisten eine möglichst umfassende Abdeckung der Umgebung. Der HCSR04-Ultraschallsensor hat einen Messbereich von 2 cm bis 400 cm und eine Genauigkeit von ±3 mm. Er operiert mit einer Spannung von 5V, benötigt 15 mA und misst Entfernungen über die Zeitdifferenz zwischen ausgesendetem und reflektiertem Ultraschallimpuls. Die Ultraschallsensoren sind dafür vorgesehen, Hindernisse in unmittelbarer Nähe des Fahrzeugs zu erkennen. Diese Funktion ist jedoch derzeit noch nicht in die Fahrzeugsoftware integriert.

TODO THEO UND MATTHES: Sensoren sind nicht angeschlossen. Erwähnen wieso so angeordnet z.B. mit Abstrahlwinkel in Skizze und zeitlich versetztes Senden um 'Überlappen' der Signale zu verhindern. Hier einfach Krolldo fragen. Der hat dafür ne Matlab Demo geschrieben. Die können wir noch nutzen

2.3.3 Kompass

Zur Bestimmung der absoluten Orientierung des Fahrzeugs wird ein zentral eingebauter Kompass verwendet. Es handelt es sich dabei um einen Adafruit

BNO055. Der Theorie nach eignet sich der Sensor gut für kurz- und mittelfristige Winkelmessungen, leidet jedoch bei Langzeitanwendungen an Driftfehlern, die durch die Integration von Gyroskopdaten entstehen können. Zwar tragen die Magnetometerwerte dazu bei, diesen Drift zu korrigieren, sie sind jedoch empfindlich gegenüber magnetischen Störungen, was vor allem in Innenräumen zu Ungenauigkeiten führen kann. Wir konnten bei unseren Anwendungen keine mit der Zeit größer werdenden Ungenauigkeiten feststellen.

2.3.4 Lidar

Beim dem Ultra-Puck VLP-32C von Velodyne handelt es sich um einen 3D-Scanning Lidar-Sensor, der durch einen sich motorisch drehenden Spiegel ein horizontales Sichtfeld von 360° erreicht. Die Reichweite des Sensors beträgt 200 m und das vertikale Sichtfeld 40°, wobei 25° vom Mittelpunkt nach unten und 15° nach oben erreichbar sind. Der LiDAR hat 32 Kanäle und eine typische Genauigkeit bei der Distanzmessung von ±3 cm.

Das Datenblatt mit allen weiteren technischen Daten kann hier über die Hochschulcloud oder hier über eine Händlerwebsite eingesehen werden.

2.4 Sicherheitsfunktionen

Um die Sicherheit des Fahrzeugs während des Betriebs zu gewährleisten, wurden umfassende Sicherheitsfunktionen integriert. Diese umfassen eine Funk-Notstoppfunktion, die das Fahrzeug sofort anhält, sobald mit einer Fernbedienung ein Signal ausgelöst wird, sowie Endanschläge, die verhindern, dass die Lenkwinkel der Räder ihre vorgegebenen Grenzen überschreiten. Die Endanschläge werden nur aktiviert, wenn die Schrittmotoren ungewollter Weise zu viele Schritte ausführen. Softwaretechnisch tritt dieser Fall nicht ein, da durch die Programmierung die maximalen Lenkwinkel auf 95° beschränkt sind. Dennoch fahren die Schrittmotoren gelegentlich falsche Winkel an. Die Ursachen dafür wurden nicht gefunden.

Für die Endanschläge sind an jedem Rad zwei ME-8108 Rollenschalter als Grenztaster für ca. +100° und -100° verbaut, die bei Überdrehung einen Stromkreis unterbrechen - also einen Schalter öffnen.

Alle acht Endanschläge sind in Reihe geschaltet und werden mit 5 V versorgt. Der Spannungswert wird auf einem Interrupt-fähigen Pin des Arduino 1 während der Nutzung überprüft. Sobald ein Endanschlag ausgelöst und damit ein Schalter geöffnet wird, fällt der Wert der Spannung von 5V auf 0V ab. Es wird ein Hardware-Interrupt ausgelöst, welcher das Auto anhält und alle Räder auf die Nullposition steuert. Die Interrupt Service Routine (ISR) selbst setzt nur eine Flag des Datentyps `volatile bool` auf `true`. Diese Flag wird regelmäßig bei jedem Durchlauf des Loops überprüft und sollte sie `true` sein, werden entsprechende Befehle an die VESCs und den Arduino 2 gesendet. Die ISR ist absichtlich kurz, um durch diesen Interrupt nicht den Timer-Interrupt für das Senden an MATLAB zu stören. Die Flag muss jedoch als `volatile` deklariert werden, um sie immer aus dem Speicher zu lesen, ansonsten könnte im Cache oder den Registern noch ein alter Wert gespeichert sein.

Die Notstoppfunktionen sind in ihrer Funktionsweise den Endanschlägen sehr ähnlich. Sie können entweder durch das Drücken eines Not-Aus-Knopfes am Fahrzeug oder über einen Funk-Not-Aus-Schalter aktiviert werden. Sowohl der Not-Aus-Knopf als auch der Funkempfänger sind in Reihe geschaltet und werden mit 5 V versorgt. Sie fungieren als elektrische Schalter, die sich bei Auslösung öffnen. Die Spannung wird, ähnlich wie bei den Endanschlägen, über einen Interrupt-fähigen Pin des Arduino 1 überwacht. Bei einer wechselnden Flanke wird die Not-Stopp-ISR aktiviert, die das Fahrzeug stoppt und den Arduino in eine leere Endlosschleife versetzt. Die Software Umsetzung der ISR ist analog zu der für die Endanschläge. Um das Fahrzeug erneut in Betrieb zu nehmen, muss der Arduino-Code neu geladen werden. Im Gegensatz zum Verhalten bei den Endanschlägen bleiben die Lenkwinkel in der zuletzt eingestellten Position und werden nicht auf 0° zurückgesetzt um den Betrieb schnellstmöglich zu beenden.

3 Software

Die folgenden Abschnitte erklären die drei eigenständigen Softwarekomponenten die im Rahmen des Projekts entwickelt wurden:

- Xbox-Controller Fahrzeugsteuerung
- Autonome Fahzeugsteuerung
- Kalibrierung der Absolutwertgeber

Mit der ersten Komponente ist es möglich, das Auto ferngesteuert durch eine unbekannte Umgebung zu manövrieren und dabei zu kartieren. Anschließend kann mit der zweiten Komponente eine Trajektorie innerhalb dieser Karte geplant und abgefahren werden. Die dritte Komponente dient der Kalibrierung und ist eventuell nützlich für zukünftige Arbeiten am Auto.

Referenzen auf Codezeilen beziehen sich auf den Anhang oder die dazu identischen Quelldateien. Der Code für den Arduino 2 ist für die Xbox-Controller Steuerung und die autonome Fahrt identisch, aber zur Übersichtlichkeit zweimal eingefügt. Die Datei `VescNimbulus.cpp` stammt aus der Arbeit von Thomas Zimmerman [7] und ist auch immer identisch.

3.1 Xbox-Controller Fahrzeugsteuerung

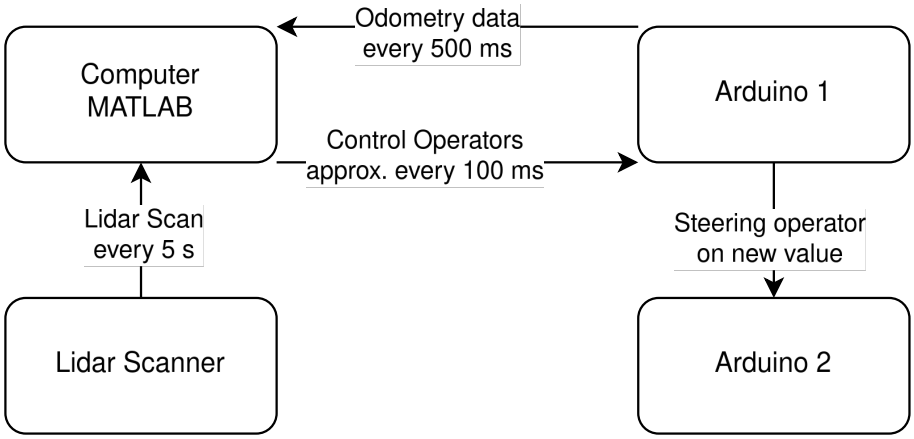
Dieser Teil der Software besteht aus zwei Quelldateien für den Arduino 1 (`ardu1_xbox_remote.ino`, `VescNimbulus.cpp`), einer für den Arduino 2 (`ardu2_xbox_remote.ino`) und zwei für MATLAB (`matlab_xbox_remote.m`, `odometryToRelPose.m`). Er ermöglicht es, einen Xbox-Controller an den PC anzuschließen und damit den Nimbulus-E zu steuern. Es sind nur translative Bewegungen oder Rotationen um den geometrischen Schwerpunkt möglich. Je nach Wunsch, kann der Lidar genutzt werden, um während der Fahrt eine Karte aufzunehmen.

Im MATLAB Code `matlab_xbox_remote.m` befinden sich einige Kommentare, die mit `% CHANGE:` anfangen. An diesen Stellen kann oder muss eine Änderung für den jeweiligen Durchlauf vorgenommen werden z. B. kann man in Zeile 18 festlegen, ob die gesammelten Daten des Durchlaufs (Odometriewerte des Autos, daraus berechnete Trajektorie, SLAM-Objekt) gespeichert werden

sollen oder nicht und in Zeile 55 der korrekte Serielle Port zum Arduino 1 gesetzt werden.

3.1.1 Kommunikation während Xbox-Controller Steuerung

Das folgende Diagramm veranschaulicht, wann welche Daten verschickt werden:



Der MATLAB-Code ist die meiste Zeit in einer Endlosschleife. In dieser wird zuerst überprüft, ob ein neues Bild für den SLAM aufgenommen werden soll, dann ob Daten vom Arduino 1 vorhanden sind und anschließend werden die Eingaben vom Xbox-Controller gelesen und die Befehle an den Arduino 1 entsprechend gesetzt und am Ende der Schleife gesendet. Es existiert keine exakte Zeitsteuerung und das Empfangen von Daten vom Arduino 1 und Updaten des SLAM mit neuen Bildern stellt jeweils eine Verzögerung dar, die Steuerung hat sich jedoch trotzdem als ausreichend handhabbar erwiesen.

Der von MATLAB gesendete String folgt dem Schema

```
<'steeringCommand', 'steeringAngle', 'rotRate'>
```

wobei die spitzen Klammern notwendig sind, um den String auf dem Arduino 1 korrekt zu parsen. Die Werte für `steeringCommand`, `steeringAngle` und `rotRate` werden entsprechend Kapitel 3.1.2 zuvor zugewiesen. Der String wird dann in Zeile 273 zusammengesetzt, in Zeile 275 an den Arduino 1 gesendet. Die anschließende Pause ist notwendig, um einen Bufferüberlauf im Arduino 1 zu verhindern.

Das empfangende Gegenstück auf der Seite des Arduino 1 besteht aus zwei Schritten. Erst werden Daten zwischen < und > empfangen, dann wird der gelesene String wieder in die Bestandteile geparst.

Empfangen wird Byte für Byte, solange Daten im Buffer verfügbar sind. Tritt eine Pause in der Übertragung auf, wird die Funktion `receiveSerialData()` verlassen. Somit wird ermöglicht, andere Aktionen auf dem Arduino 1 auszuführen wie zum Beispiel das Überprüfen der Safety-Checks oder das Senden der Geschwindigkeitswerte an die VESCs, sodass diese nicht in den Ruhemodus wechseln. Bei neuen Daten wird `receiveSerialData()` das Empfangen fortsetzen und durch die globalen Variablen den Zustand des vorherigen Aufrufs übernehmen. Erst wenn die Nachricht vollständig empfangen wurde (ein > wird gelesen), werden die Zustandsvariablen zurückgesetzt. Die Flag `newData` signalisiert, dass der empfangene String nun geparst werden muss.

Die Drehzahl wird direkt übernommen (falls nicht durch `steeringCommand` überschrieben) und in Zeile 178 am Ende des Loops immer vom Arduino 1 an die VESCs versendet. Wenn die VESCs zu lange kein Signal erhalten, schalten sie ab, weshalb dieser Befehl jedes Mal im Loop ausgeführt wird.

Die Steuerbefehle werden im Enum `steeringCmd` definiert und in der Funktion `handleTurningCmds()` behandelt.

```
66 enum steeringCmd {
67   NO = 0,    // do not change steering angle
68   YES = 1,   // change steering angle
69   TURN = 2, // enter "turn on the spot" mode
70   POS = 3,   // rotate math postive in "turn on the spot"
71   NEG = 4,   // rotate math negative in "turn on the spot"
72   END = 5,   // end program (rps = 0, angle = 0, endless loop)
73 };
```

ardu1_xbox_remote.ino

Bei `NO` soll nur die Drehzahl angepasst werden. Dadurch wird bei nicht geänderter Richtung verhindert, dass die Reifen wegen der Ungenauigkeit der Winkelsensoren hin und her zittern. `YES` schickt lediglich vier Mal den gleichen Winkel an den Arduino 2, `TURN` aktiviert das „auf der Stelle drehen“, indem die Reifen auf $\pm 57°$ eingestellt werden und dementsprechend mit der Achse zum geometrischen Mittelpunkt zeigen. Auch dafür wird eine Nachricht

mit den passenden Winkeln an den Arduino 2 gesendet, außerdem werden alle Drehzahlen mit 0 überschrieben. `POS` und `NEG` setzen die Drehzahlen aller Räder auf ±1 Umdrehung pro Sekunde, sodass sich das Auto mathematisch positiv (gegen den Uhrzeigersinn) bzw. mathematisch negativ (im Uhrzeigersinn) dreht. Der Befehl sollte also nur nach einem `TURN` geschickt werden. `END` sendet Drehzahl 0 an alle VESCs und Winkel 0° für alle Reifen an den Arduino 2 und endet dann in einer Endlosschleife.

Abgesehen vom Auslösen der Endanschläge sind das alle Stellen, an denen der Arduino 1 Lenkwinkel an den Arduino 2 schickt. Das Empfangen beim Arduino 2 ist nahezu identisch mit dem vom Arduino 1, mit dem Unterschied beim Parsen. Der Arduino 2 empfängt Nachrichten immer im Format[1]

```
<'steeringAngleVL', 'steeringAngleVR',
 'steeringAngleHL', 'steeringAngleHR'>
```

Alle 500 Millisekunden liest der Arduino 1 die Odometriedaten der VESCs, der Winkelsensoren an den Reifen und aus dem Kompass und schickt diese an MATLAB. Die Lenkwinkel werden dabei alle vier gemittelt, die Drehzahlen auch, allerdings nur im Betrag. Die Richtung wird dem Reifen vorne rechts entnommen. Andernfalls addierten sich die Drehzahlen immer zu 0 im „auf der Stelle drehen“ Modus. Gesendet wird eine Nachricht im Format

```
<'timestamp', 'steeringAngle', 'rotationRate', 'psi'>
```

In MATLAB wird der String mit `readline()` gelesen und dann direkt geparst, die Kommunikation ist also blockierend implementiert. Die Odometriedaten werden anschließend genutzt um mittels der inversen Kinematik die zurückgelegte Trajektorie zu bestimmen, siehe Kapitel 3.1.3.

3.1.2 Steuerung mit dem Xbox-Controller

Die Steuerung des Fahrzeugs mit dem Xbox-Controller wurde implementiert, um dem Benutzer eine manuelle Kontrolle über das Fahrzeug zu ermöglichen. Trotz möglicher acht Stellgrößen ist für eine intuitivere Bedienung eine vereinfachte Steuerform implementiert. Es ist entweder eine rein translatorische oder

[1] Zeilenumbruch nur zur Lesbarkeit

eine rein rotatorische Bewegung des Fahrzeugs möglich. Das Fahrzeug kann sich somit linear fortbewegen (alle Lenkwinkel und Drehzahlen sind gleich) oder sich auf der Stelle drehen.

Die Lenkwinkel der Reifen sind standardmäßig auf 0° gesetzt, wenn die Räder geradeaus ausgerichtet sind. Durch Verkabelungen sind sie auf +90° bzw. −90° begrenzt. Sollte mit dem Joystick ein Lenkwinkel über 90° vorgegeben werden, wird dieser auf den entsprechenden Winkel im negativen Bereich gemappt und die Drehzahl umgekehrt, sodass das Fahrzeug weiterhin in die gewünschte Richtung fährt. Ein Beispiel ist eine vorgegebene Drehzahl von einer Umdrehung pro Sekunde mit einem Lenkwinkel von 100°, die auf eine Drehzahl von minus einer Umdrehung pro Sekunde mit einem Lenkwinkel von −80° umgerechnet wird. Das Fahrzeug steuert somit trotz Lenkwinkelbeschränkung von +90° bzw. −90° in die vom Joystick vorgegebenen Richtung von 100°.

Die Steuerungsmodi des Fahrzeugs lassen sich mit den Tasten des Controllers verändern. Im translatorischen Modus wird der linke Joystick genutzt, um die Richtung anzugeben. Die Tasten `RT` oder `LT` bestimmen die Geschwindigkeit. Dabei sind Drehzahlen zwischen 0,15 und 1,3 möglich. Wird die Taste `A` gedrückt, werden alle Räder auf den Lenkwinkel von 0° zurückgesetzt, was eine geradeaus gerichtete Fahrt ermöglicht. Wird die Taste `X` gedrückt, wird das Steuerungsprogramm beendet. Drückt der Benutzer die Taste `Y`, wechselt das Fahrzeug in den rotatorischen Modus, bei dem sich das Fahrzeug auf der Stelle dreht. In diesem Modus werden die Lenkwinkel der Räder so eingestellt, dass der Momentanpol (vgl. Kap. 3.2.4) in der Mitte des Fahrzeugs liegt. Die Räder vorne links und hinten rechts werden auf −57° gesetzt, während die Räder vorne rechts und hinten links auf +57° eingestellt werden. So wird eine Drehung des Fahrzeugs um seine eigene Achse ermöglicht.

Um die Drehrichtung zu steuern, können die Tasten `RB` und `LB` verwendet werden. Drückt der Benutzer die Taste `RB`, dreht sich das Fahrzeug im Uhrzeigersinn, wobei die Drehzahl der Räder auf eine Umdrehung pro Sekunde eingestellt wird. Wird die Taste `LB` gedrückt, dreht sich das Fahrzeug mit gleicher, aber entgegengesetzter Raddrehzahl gegen den Uhrzeigersinn. Drückt der Benutzer erneut die Taste `A`, werden alle Räder auf den Lenkwinkel von 0° gesetzt. Das Fahrzeug wechselt zurück in den translatorischen Modus.

Eine visuelle Darstellungen der Controllertasten samt ihrer Bedeutungen findet sich in Abb. 2

Im Code ist die Abfrage nach der gedrückten Taste als lange Kaskade von if-else Verzweigungen implementiert. Die Reihenfolge der Statements ist dabei insofern wichtig, als dass weiter oben aufgeführte Tasten die späteren überschreiben. Das Drücken von `A` und `RT` gleichzeitig bewirkt z.B. nur das Nullstellen der Reifen, keine Drehzahl. Außerdem ist die Reihenfolge auch für die korrekte Zustandsbehandlung translatorisch oder rotatorisch notwendig.

Während der Xbox-Controller Steuerung gibt es die Möglichkeit, die Umgebung mit einem Lidar aufzunehmen und mit dem SLAM Algorithmus eine 2D-Hinderniskarte der abgefahrenen Umgebung zu erstellen. Eine detaillierte Erklärung des SLAM-Algorithmus findet sich in Kapitel 3.1.4. Die Kommunikation zwischen MATLAB, dem Arduino 1 und dem Lidar wird in Kapitel 3.1.1 genauer erläutert.

Wichtiger Hinweis bei auftretenden Problemen:
Durch einige parallel laufende Studienarbeiten wurden leider durch erneut fehlerhaftes Bedienen des Fahrzeuges ein oder mehrere Endanschläge beschädigt. Ein Wackelkontakt öffnet kurzzeitig den Stromkreis, wodurch der Endanschlaginterrupt und die damit verbundene Interrupt-Service-Rountine ausgeführt wird. Sollte dies während der rein rotatorischen Steuerung erfolgen, werden die Lenkwinkel auf 0° gesetzt, der Drehmodus jedoch nicht verlassen. Um die Steuerung weiterhin wie gewollt nutzen zu können, muss deshalb durch Drücken der Teste `A` zuerst in den translatorischen Modus überführt werden.

3.1.3 Inverse Kinematik bei Xbox-Controller Steuerung

Die während der Xbox-Controller Steuerung von dem Fahrzeug tatsächlich zurückgelegte Strecke, wird über inverse Kinematik berechnet und aufgezeichnet. Die Position und Ausrichtung des Fahrzeugs werden während der Kartenerstellung dem SLAM-Algorithmus als zusätzliche Informationen übergeben.

Während der Steuerung mit dem Xbox-Controller wird das Fahrzeug rein translatorisch oder rein rotatorisch bewegt. Das bedeutet im rein translatorischen Fall, dass alle Lenkwinkel und alle Drehzahlen gleich sind und das Auto

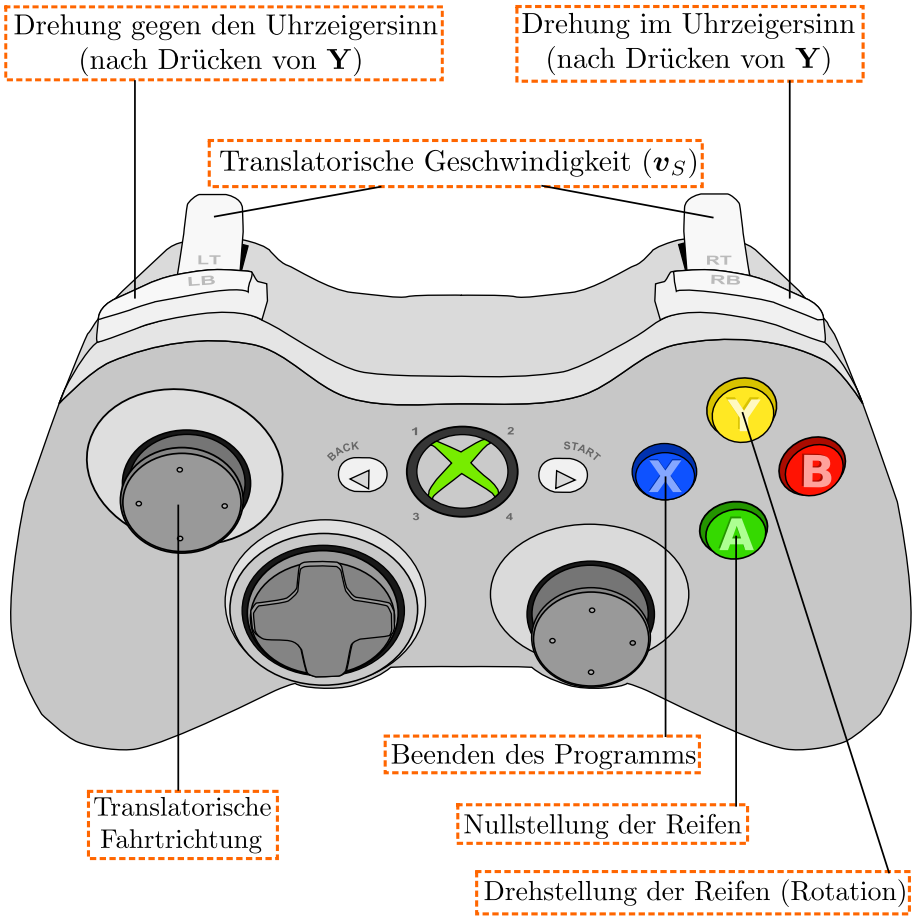


Abbildung 2: Xbox-Controller und Tastenbedeutungen

seine Ausrichtung über die Zeit hinweg theoretisch nicht ändert. Durch Ungenauigkeiten der Lenkwinkel ist die Annahme der unveränderten Ausrichtung in der Realität nicht immer korrekt.

Im rein rotatorischen Fall, dreht das Auto sich auf der Stelle. Die Lenkwinkel und Drezahlen sind derart bestimmt, dass der Momentanpol (vgl. Kap. 3.2.4 zur Erklärung des Momentanpols) in der geometrischen Mitte des Fahrzeugs liegt. Die Position des Schwerpunkts des Autos (vereinfacht als geometrische Mitte angenommen) verändert sich während der Drehung nicht.

Um die zurückgelegte Trajektorie zu berechnen, werden die Lenkwinkel, Drehzahlen sowie die absolute Ausrichtung des Autos vom Arduino ausgelesen und per serieller Kommunikation an MATLAB versendet.

Die Berechnung erfolgt im Weltkoordinatensystem und basiert auf den vom Arduino ausgelesenen Daten sowie den zugehörigen Zeitstempeln. Die Funktion zur inversen Kinematik berechnet bei translatorischer Fahrt die Änderungen in der x- und y-Position (Δx und Δy), die in der Hauptdatei zu den absoluten Positionen im Weltkoordinatensystem addiert werden. Die absolute Ausrichtung wird durch die bereits erwähnten Ungenauigkeiten der Lenkwinkel auch bei translatorischen Fahrten über die Rückgabe des Kompasses stetig aktualisiert.

Die Berechnung der Änderungen in der x- und y-Position basiert auf der Formel $v = \omega \cdot R = 2\pi n \cdot R$, wobei n die Drehzahl und R der Reifenradius ist. Die Drehzahl wird über die Zeitdifferenz nach $N = n \cdot \Delta t$ integriert, um die von den Rädern zurückgelegte Strecke zu ermitteln, vgl. (1) und (2).

Durch Addition von aktueller Ausrichtung des Fahrzeugs ψ und aktuellem Lenkwinkel aller Räder δ und $\alpha = \psi + \delta$, kann die Richtung bestimmt werden, in die das Auto im Weltkoordinatensystem translatorisch steuert. Die Formeln zur Berechnung von Δx und Δy lauten:

$$\Delta x = cos\left(\frac{\alpha \cdot \pi}{180}\right) \cdot 2\pi N \cdot R \tag{1}$$

$$\Delta y = sin\left(\frac{\alpha \cdot \pi}{180}\right) \cdot 2\pi N \cdot R \tag{2}$$

Wird das Fahrzeug nur auf der Stelle gedreht, bewegt es sich nicht translatorisch. Δx und Δy sind gleich Null. Die Ausrichtung ψ wird weiterhin direkt

vom Kompass des Arduino ausgelesen und in MATLAB gespeichert.

Dieser Ansatz zur inversen Kinematik ermöglicht eine präzise Berechnung der unter der Xbox-Controller Steuerung zurückgelegten Trajektorie, die für den SLAM-Algorithmus von entscheidender Bedeutung ist.

3.1.4 SLAM-Algorithmus

Das Scannen der Umgebung mit dem LIDAR Sensor und Kartieren mithilfe des Simultaneous Localization And Mapping (SLAM) Algorithmus baut auf der Arbeit von Niklas Schlosser[8] auf. Die generelle Vorgehensweise wurde nicht verändert. Es wird zuerst der letzte Scan vom Lidar gelesen, dann vorbereitet (Hohe Punkte und Boden entfernen, Gesamtanzahl an Punkten zufällig runter rechnen, Ausreißer entfernen, 3D auf 2D Projektion) und schließlich dem für den SLAM verantwortlichen Objekt übergeben.

Einige Parameter in der Vorbereitung wurden an unsere Situation angepasst. Die wichtigsten Veränderungen sind

- Online Ausführung: Die Scans werden direkt während der Fahrt verarbeitet und die Karte mit jedem neuen Scan aktualisiert. Alle weiteren Änderungen waren aufgrund dieser geänderten Anforderung notwendig, um zu garantieren, dass die Steuerung weiterhin flüssig läuft.
- Odometrie für relative Posen: Der SLAM kann potenziell die relative Bewegung zwischen zwei Scans durch Computervision Tools schätzen. Durch die Bereitstellung einer eigenen Schätzung basierend auf den Odometriedaten des Fahrzeugs konnte aber eine signifikante Rechenzeitreduzierung um den Faktor 10 erreicht werden.[2].
- Parallelisieren: Die MATLAB Parallel Computing Toolbox wird genutzt, um die Aufbereitung der Scans und die SLAM-Verarbeitung parallel zur Steuerung über den Xbox-Controller auszuführen. Dazu wird ein Pool mit einem parallelen Prozess erstellt. So kann dieser Schritt bis

[2]Getestet mit den Daten von Niklas Schlosser [8] im offline Betrieb und der dortigen SLAM Variante - ein konkreter Vergleich zur finalen Version ist daher nicht mehr sinnvoll, weil z. B. die Schleifenschlusserkennung und Optimierung in den beiden Ansätzen unterschiedlich behandelt wird.

zu `slamInterval` Sekunden dauern, ohne Verzögerungen in der Steuerung zu verursachen. Bisher haben 5 Sekunden ausgereicht, damit die SLAM-Verarbeitung rechtzeitig abgeschlossen war und die Bilder nah genug zusammenliegen, um eine gute Kartierung zu gewährleisten. Dafür musste jedoch die komplette Verarbeitung der Scans in eine Funktion ausgelagert werden, die dem parallelen Worker übergeben wird.

- Andere SLAM Variante: `lidarscanmap` aus der MATLAB Lidar Toolbox wurde durch `lidarSLAM` aus der MATLAB Navigation Toolbox ersetzt, weil zum Einen die Implementierung etwas freundlicher gestaltet ist, z. B. wird die komplette Schleifenschlusserkennung und damit verbundene Optimierung in der `addScan` Funktion versteckt. Zum Anderen wird ein Branch and Bound Algorithmus zur Registrierung eines neuen Scans (also die Schätzung der relativen Pose) genutzt. Dieser verwendet die extern geschätzte relative Pose als Ausgangspunkt, um die Ergebnisse zu verbessern, bezieht aber trotzdem auch die optischen Informationen der beiden Scans mit ein und ist damit in unserer Erfahrung präziser, als nur die durch Odometrie geschätzte relative Pose zu nutzen.

Die Nutzung des Lidars kann in Zeile 16 des Programms `matlab_xbox_remote.m` aktiviert werden. Ohne Lidar ist das Programm nützlich, um das Auto an einen anderen Ort zu bewegen. Mit Lidar wird automatisch kartiert, beim ersten Starten des Programms oder nach einer Pause von mehr als 30 Minuten muss allerdings der parallel Pool neu gestartet werden, was einige Minuten dauert.

3.2 Autonome Fahrzeugsteuerung

Die autonome Fahrzeugsteuerung ist der Endpunkt der Softwareentwicklung. Sie nutzt die Informationen aus der 2D Belegungskarte und der Lokalisierung des Fahrzeugs, um eine Trajektorie zu planen und diese abzufahren. Die Komponente besteht aus je zwei Quelldateien für den Arduino 1 (`ardu1_autonomous.ino`, `VescNimbulus.cpp`), einer für den Arduino 2 (`ardu2_autonomous.ino`) und insgesamt fünf Dateien für MATLAB (`matlab_autonomous.m`, `findNearestTan.m`, `odometry3DOFtoRelPose.m`, `pathPlanningHybridAStar.m`, `trajectoryTo8DOF.m`). Zusätzlich werden vier

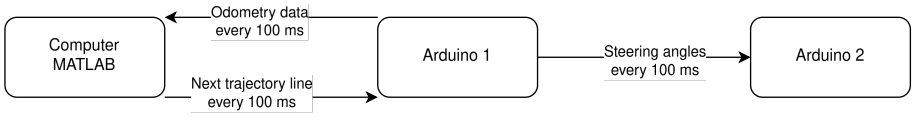
Beispieltrajektorien bereitgestellt, welche in den Zeilen 15 bis 18 von `matlab_autonomous.m` gewählt werden können:

- **Circle:** Da Auto **fährt** in konstanter Geschwindigkeit einen Kreis mit vorgegebenem Radius ab - in `buildCircleTrajectory.m`
- **Rotating:** Das Auto dreht sich auf der Stelle hin und her bis zu einem vorgegebenen Winkel und für gegebenen Anzahl an Durchläufen in `buildRotatingTrajectory.m`
- **Rotating and Moving:** Wie Rotating nur mit Vorwärtsbewegung um gegebene Anzahl an Metern in `buildRotatingAndMovingTrajectory.m`
- **From Occupation Map:** Eine Trajektorie wird in eine gegebene Belegungskarte mit gegebener Start- und Zielpose berechnet. Dafür muss ein Dateiname übergeben werden, unter welchem die Belegungskarte mit dem Namen `occMap` gespeichert wird. Eine Beispielhafte Belegungskarte aus dem Automotive Center der THU ist unter dem Namen `occMap.mat` verfügbar.

Die Parameter für alle Varianten können in den Zeilen 38 ff. gesetzt werden. Des Weiteren finden sich Kommentare, die mit `% CHANGE:` anfangen - hier können oder müssen Anpassungen vorgenommen werden wie z.B. in Zeile 23 der richtige Serielle Port zum Arduino 1 gewählt werden.

3.2.1 Kommunikation während autonomer Fahrzeugsteuerung

Das folgende Diagramm veranschaulicht, wann welche Daten verschickt werden:



Die Zeitschritte können auch geändert werden, müssen dann aber in allen drei Hauptquelldateien identisch sein. Zu große Zeitschritte führen zu großen Fehlern in der Diskretisierung der Trajektorie und sind dadurch nicht möglich, 500ms waren bei der Cirlce Trajektorie schon ausreichend um mit bloßem Auge falsche Lenkwinkel zu erkennen.

Die Zeitsteuerung basiert auf einem Timer-Interrupt auf dem Arduino 1. Dieser löst das Senden aller Odometrieinformationen an MATLAB aus. Gesendet werden der aktuelle Zeitstempel, der Gierwinkel, die vier Lenkwinkel und die vier Drehzahlen in der Form

```
<'timestamp', 'psi',
'angleVL', 'angleVR', 'angleHL', 'angleHR',
'rotRateVL', 'rotRateVR', 'rotRateHL', 'rotRateHR'>
```

Sobald MATLAB diese Informationen erhalten hat, wird die nächste Zeile der Trajektorie versandt und danach die Odometriedaten zur Bestimmung der tatsächlichen Trajektorie genutzt, siehe Kapitel 3.2.5. Der Arduino 1 leitet dann noch die erhaltenen Lenkwinkel an den Arduino 2 weiter.

Der String von MATLAB zum Arduino 1 mit den Zeilen der Trajektorie folgt dem Schema

```
<'angleVL', 'angleVR', 'angleHL', 'angleHR',
'rotRateVL', 'rotRateVR', 'rotRateHL', 'rotRateHR'>
```

Der String vom Arduino 1 zum Arduino 2 mit den Lenkwinkel dem Schema

```
<'angleVL', 'angleVR', 'angleHL', 'angleHR'>
```

Die Implementierung des Sendens und Empfangens ist analog zu dem in Kapitel 3.1.1 erklärten Code.

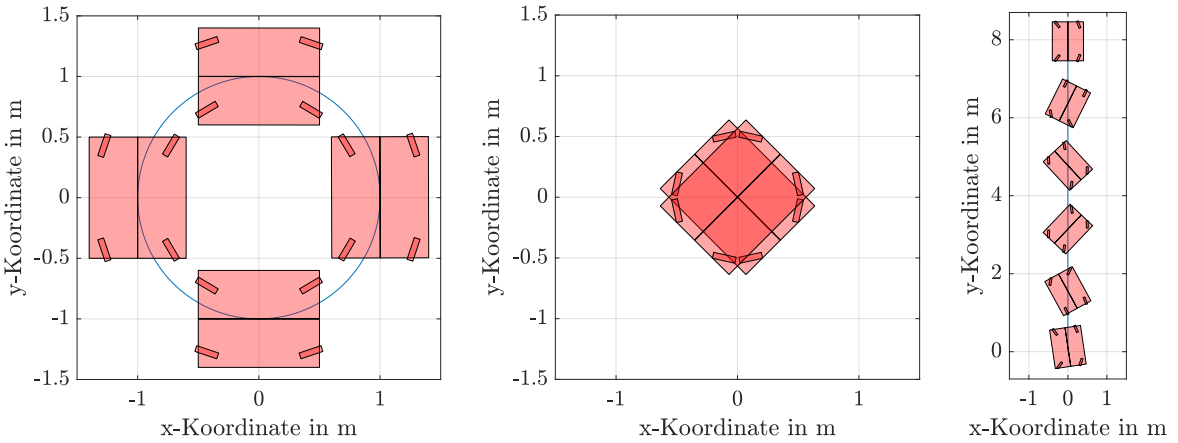
3.2.2 Berechnung der Testtrajektorien

In einem ersten Schritt der autonomen Steuerung wurden Trajektorien nicht in einer zuvor aufgenommenen Karte der Umgebung mit speziellen Pfadplanungsalgorithmen gefunden (vgl. Kap. 3.2.3), sondern künstlich erzeugt und vorgegeben. Somit konnte überprüft werden, ob die Implementierung der in Kapitel 3.2.4 errechneten Lenkwinkel und Drehzahlen richtig und funktionstüchtig war. Dafür wurden die Größen $t, x, y, \psi, \dot{x}, \dot{y},$ und $\dot{\psi}$ entsprechend der

jeweiligen Trajektorien erzeugt. Anschließend wurde das Auto samt seiner eingeschlagenen Räder über die Abfahrt der Trajektorie hinweg in einem MATLAB-Plot simuliert und später in der Realität mit den vorgegebenen Größen angesteuert und fahren gelassen.

Es wurden drei Testtrajektorien erzeugt:

1. **Kreisfahrt:** Das Auto fährt eine Runde auf einem Kreis. Radius und Geschwindigkeit können angepasst werden, vgl. Abb. 3a.
2. **„Schwanken“:** Das Auto dreht sich auf der Stelle, periodisch von links nach rechts. Amplitude (maximaler Drehwinkel) sowie Periodendauer können vorgegeben werden. Ort und Geschwindigkeit sind durch trigonometrische Funktionen bestimmt. Eine Beispielabbildung dieser Trajektorie findet sich in Abb. 3b.
3. **„Schwankende“ Geradeausfahrt:** Das Auto fährt eine gerade Linie nach vorne. Währenddessen wird die Bewegung von der in 2. beschriebenen Schwankbewegung überlagert. Eine Beispielabbildung dieser Trajektorie findet sich in Abb. 3c.



(a) Kreisbahn **(b)** Schwanken **(c)** Geradlienig

Abbildung 3: Testtrajektorien

3.2.3 Berechnung der Trajektorie

Die Berechnung der Trajektorie durch eine zuvor aufgenommene Umgebung ist ein zentraler Aspekt der autonomen Steuerung des Fahrzeugs. Sie basiert

auf dem „Hybrid A*“-Algorithmus, der speziell für Pfadplanungen in Fahrzeuganwendungen entwickelt wurde. Der Algorithmus kombiniert die Vorzüge des klassischen A*-Algorithmus mit einer Pfadplanung für nicht-holonome Systeme, wodurch Rückwärtsbewegungen und enge Wendungen möglich sind. Dieser Algorithmus berücksichtigt die Minimalwenderadien und die Bewegungseinschränkungen von Fahrzeugen und ist besonders für die Trajektorienplanung in Umgebungen mit Hindernissen geeignet. [**<empty citation>**]
TODO THEO: cite einfügen

Der Hybrid A* Algorithmus ist Teil der MATLAB Navigation-Toolbox, welche eine Reihe von Funktionen und Klassen zur autonomen Steuerung von Robotern bietet.

Zu Beginn der Trajektorienberechnung wird eine zuvor mit dem Lidar aufgenommene Belegungskarte aus einer separaten Datei geladen. Anschließend wird über einen Threshhold eine binären Belegungskarte erstellt. Der Threshhold wurde in unserem Fall experimentell festgelegt und kann je nach Umgebungsbedingungen verändert werden. Die binäre Belegungskarte repräsentiert die Umgebung des Fahrzeugs samt freier und belegter Flächen und wird vom Hybrid A* Algorithmus verwendet, um eine optimale Trajektorie zu berechnen.

Anschließend wird die Karte einem `plannerHybridAStar`-Objekt übergeben. Zusätzlich werden fahrzeug- und hindernisspezifische Parameter festgelegt, die das Verhalten des Algorithmus steuern.

Der Hybrid A* Algorithmus berechnet mit den Kartendaten eine Trajektorie vom Start- zum Zielzustand, welche dabei in Form von t, x, y, ψ definiert werden, wobei ψ die Orientierung des Fahrzeugs in Bezug auf die x-Achse angibt. Die errechnete Trajektorie wird in einer Reihe von Zuständen ausgedrückt, die wie die Start- und Endpunkte die Position und Orientierung des Fahrzeugs zu verschiedenen Zeitpunkten repräsentieren.

Zusätzlich wird die geplante Trajektorie mittels der `optimizePath`-Funktion weiter verfeinert, um eine glattere und optimierte Fahrbahn zu generieren. In der Pfadoptimierung wird eine maximale Geschwindigkeit ($vMax$) sowie eine Zeitschrittweite ($tStep$) definiert, die sicherstellen, dass der Wagen in einem realistischen Geschwindigkeitsbereich fährt und Hindernissen mit einem Sicherheitsabstand ausweicht.

Im letzten Schritt der Trajektorienplanung werden aus Ort und Orientierungsinformationen zu bestimmten Zeitpunkten die Geschwindigkeiten bestimmt. Die aus der `kineticInfo.Velocity`, einem Rückgabewert der `optimzePath`-Funktion, erhaltene Gesamtgeschwindigkeit wird mittels *sinus, cosinus* und ψ in $\dot{x}$ und $\dot{y}$ zerlegt. Analog wird $\dot{\psi}$ mit `kineticInfo.AngularVelocity` zurückgegeben.

Mit einem linearen Zusammenhang werden anschließend Ort, Orientierung und Geschwindigkeiten der errechneten Trajektorie zu den spezifischen Zeiten interpoliert, zu denen der Arduino einen zeitgesteuerten Interrupt auslöst, um echtzeitfähig die Trajektorie abzufahren. Wir nutzen eine Frequenz von 10 Hz, in denen MATLAB Trajektoriendaten an den Arduino sendet und der Arduino Ist-Lenkwinkel und Ist-Drehzahlen zurück an MATLAB.

Ein Beispiel für die Visualisierung des optimierten Pfads ist in Abbildung 6 dargestellt.

Die Details des Hybrid A* Algorithmus und seiner Implementierung sind in einer anderen Studienarbeit ausführlich beschrieben, auf die hier verwiesen wird [9].

3.2.4 Umrechnung in Lenkwinkel und Drehzahlen

Die geplante Trajektorie (t, x, y, ψ) wird in MATLAB in Lenkwinkel und Motorendrehzahlen umgerechnet, die anschließend von den Controllern angesteuert werden können.

Die Grundlage für diese Rechnung bildet die Annahme von nicht-holonomen Bindungen - also Bedingungen, bei denen nicht nur unabhängige verallgemeinerten Koordinaten das System beschränken, sondern auch Geschwindigkeitsbedinungen vorliegen.

Holonome Bindungen liegen vor, wenn das System lediglich durch geometrische Bedingungen (z. B. x, y, ψ) beschrieben wird. Ein Beispiel wäre ein Starrkörper in einer 2D-Ebene, der durch drei verallgemeinerte Koordinaten $\mathbf{q} = [x, y, \varphi]^T$ eindeutig in seiner Position und Ausrichtung definiert ist und sich in alle Richtungen frei bewegen kann.

Nicht-holonome Bindungen hingegen liegen vor, wenn zusätzlich Beschrän-

kungen des System vorhanden sind, deren Ursprung nicht nur in der Geometrie, sondern auch in der Geschwindigkeit liegt. Ein Beispiel für eine nichtholonome Bindung ist ein ohne Schlupf rollendes Rad. Es ist eindeutig definiert in Ort und Ausrichtung mit *x, y, psi*, zusätzlich jedoch beschränkt in der Richtung der Geschwindigkeit, die ohne Schlupf nur senkrecht zur Radachse möglich sein kann.

Nicht-holonome Bindungen werden häufig durch Ungleichungen oder Differentialgleichungen beschrieben. Im Beispiel des schlupflosen Reifens lautet die Zusatzbedingung

$$v = -\frac{\dot{x}}{sin\varphi} = \frac{\dot{y}}{cos\varphi} \tag{3}$$

$$\iff \dot{x} \cdot cos\varphi = \dot{y} \cdot sin\varphi = 0 \tag{4}$$

Die Berechnung der Lenkwinkel und Raddrehzahlen basiert auf der Annahme, dass sich das Fahrzeug samt seiner Reifen ohne Schlupf und Schräglauf auf der Fahrbahn fortbewegt. Dies ermöglicht die Konstruktion des Momentanpols. Der Momentanpol Ω ist der Ort, um den sich ein translatorisch und rotatorisch bewegender Starrkörper scheinbar nur rotatorisch bewegt. Läge der Momentanpol innerhalb der Körpergrenzen, hätte der Körper an diesem Ort die Geschwindigkeit 0. Ein Beispiel ist das Rad einer Mühle, dessen Achse, welche sich immer an derselben Stelle befindet, die Geschwindigkeit 0 besitzt, während alle anderen Punkte des Mühlrades sich tangential um diese Achse (den Momentanpol) drehen. Um den Momentanpol zeichnerisch zu konstruieren, müssen orthogonal zu den bekannten Geschwindigkeiten Verbindungslinien gezogen werden. Im Schnittpunkt der Verbindungslinien befindet sich der Momentanpol. Er stellt zu allen Punkten des um ihn rotierenden Starrkörpers die nicht-holonomen Geschwindigkeitsbedingungen dar. Eine Veranschaulichung des Momentanpols Ω findet sich in Abb. 4. Die blauen Vektoren $\mathbf{v}$ zeigen die Geschwindigkeitsvektoren der vier Räder und des Schwerpunktes. Die Indizes *VL, VR, HL, HR* stehen für vorne/hinten links/rechts. Die Supskripte C und W indizieren das Fahrzeugkoordinatensystem (Car) und das weltfeste System. Die Lenkwinkel sind durch δ gekennzeichnet, der Drehwinkel des Fahrzeugs durch ψ.

Um die Lenkwinkel und Raddrehzahlen zu berechnen, wird zusätzlich zu Ort

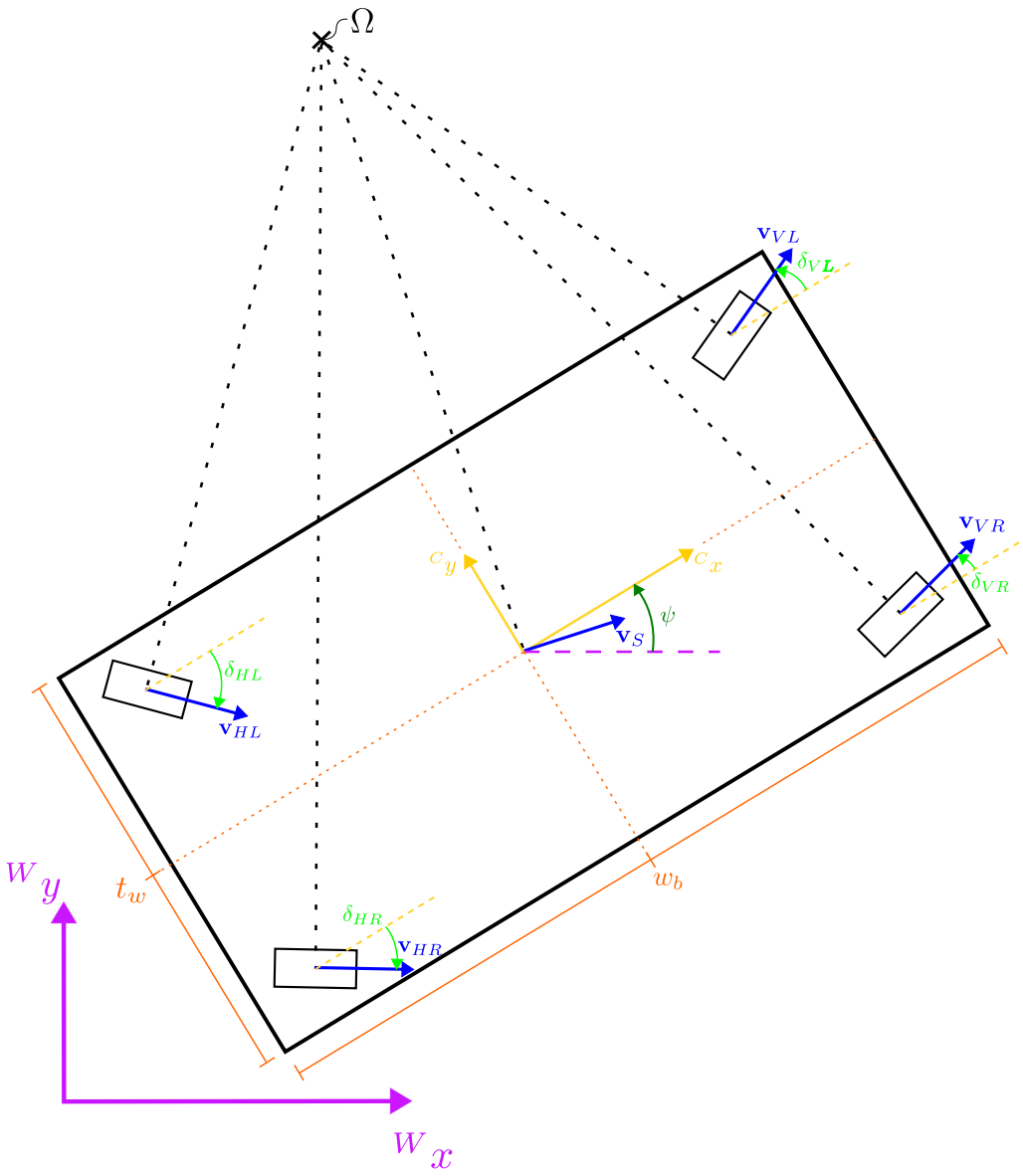


Abbildung 4: Momentanpol

und Ausrichtung auch Geschwindigkeit und Drehgeschwindigkeit des Fahrzeugs benötigt. Diese sind Teil der Hybrid-A*-Trajektorienplanung und Rückgabewert der optimierten Trajektorie, vgl. Kap. 3.2.3.

Bei Ansteuerung mit zuvor künstlich kreierten Testtrajektorien sind die Geschwindigkeiten ebenfalls Teil der Trajektorienplanung, werden hier allerdings diskret selbst berechnet, vgl. Kap. 3.2.2.

Da in beiden Trajektorienplanungsprogrammen sich die Geschwindigkeiten aus den Positionen im Weltkoordinatensystem ergeben, der Algorithmus zur Lenkwinkel- und Drehzahlberechnung jedoch im Fahrzeugkoordinatensystem stattfindet, müssen $\dot{x}$ und $\dot{y}$ noch mit ψ transformiert werden.

Das Superskript C indiziert das Fahrzeugkoordinatensystem (Car), W das Weltkoordinatensystem. Die Transformation lautet:

$$ {}^{C}\boldsymbol{v}_S = \begin{bmatrix} \cos(\psi) & \sin(\psi) & 0 \\ -\sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot {}^{W}\boldsymbol{v}_S \tag{5} $$

Durch die Annahme der nicht-holonomen Bindungen können mit Wissen über Schwerpunktsposition (x,y) und Fahrzeugausrichtung ψ sowie Schwerpunktsgeschwindigkeit $(\dot{x},\dot{y})$ und Drehgeschwindigkeit $\dot{\psi}$ des Fahrzeugs zu einem definierten Zeitpunkt t die Geschwindigkeitskomponenten (v_{Q_x}, v_{Q_y}) eines beliebigen Punktes Q des Fahrzeuges berechnet werden, sofern Kenntnis über den Verbindungsvektor zwischen Schwerpunkt S und beliebigem Punkt Q existiert. Die Formel zur Berechnung von v_Q aus v_S, $\dot{\psi}$ und r_{SQ} lautet:

$$ \boldsymbol{v}_Q = \boldsymbol{v}_S + \dot{\boldsymbol{\psi}} \times {}_S\boldsymbol{r}_Q \tag{6} $$

In Abb. 4 ist eine Skizze des Autos dargestellt, in der die Geschwindigkeiten des Schwerpunktes sowie aller Räder eingezeichnet sind. Exemplarisch ist für

das Rad vorne links die Geschwindigkeitsberechnung in Gl. (8)dargestellt.

$$\boldsymbol{v}_{VL} = \boldsymbol{v}_S + \dot{\boldsymbol{\psi}} \times {}_S\boldsymbol{r}_{VL} \tag{7}$$

$$= \begin{bmatrix} v_{S_x} \\ v_{S_y} \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} \times \begin{bmatrix} w_b/2 \\ t_w/2 \\ 0 \end{bmatrix} = \begin{bmatrix} v_{S_x} - \dot{\psi} \cdot t_w/2 \\ v_{S_y} + \dot{\psi} \cdot w_b/2 \\ 0 \end{bmatrix} \tag{8}$$

Die anderen Geschwindigkeitsvektoren ergeben sich analog zu:

$$v_{VR} = \begin{bmatrix} v_{S_x} + \dot{\psi} \cdot t_w/2 \\ v_{S_y} + \dot{\psi} \cdot w_b/2 \\ 0 \end{bmatrix}, \quad v_{HL} = \begin{bmatrix} v_{S_x} - \dot{\psi} \cdot t_w/2 \\ v_{S_y} - \dot{\psi} \cdot w_b/2 \\ 0 \end{bmatrix}, \quad v_{HR} = \begin{bmatrix} v_{S_x} + \dot{\psi} \cdot t_w/2 \\ v_{S_y} - \dot{\psi} \cdot w_b/2 \\ 0 \end{bmatrix}$$

Nachdem die Geschwindigkeitskompenenten der einzelnen Räder bestimmt wurden, können die Lenkwinkel errechnet werden. Lediglich den `arctan` zu verwenden, kann jedoch reell zu Problemen führen. Wird beispielsweise von -90 ° auf -91 ° gelenkt, gibt der `arctan` +89 ° zurück. Für diesen Fall wurde eine Funktion `findNearestTan` entwickelt, die den letzten Lenkwinkel mit aktuellen Kandidaten bei +180 °, +0 ° und -180 ° in Relation zum letzten Lenkwinkel vergleicht und den mit der geringsten Abweichung auswählt.

Um die Drehzahlen zu berechnen, wird die Formel

$$v = \omega \times r = 2\pi n \cdot r \tag{9}$$

$$\iff \quad n = \frac{v}{2\pi r} = \frac{\sqrt{\dot{x} + \dot{y}}}{2\pi r} \tag{10}$$

verwendet. Die Winkelgeschwindigkeit ω kann dabei durch $2\pi n$ ersetzt werden. n stellt in diesem Fall die Drehzahl dar. Das Kreuzprodukt entfällt in der 2D-Ebene.

Durch das Vorzeichen des Skalarprodukts von Lenkwinkelvektor $[cos(\delta) \quad sin(\delta) \quad 0]^T$ und Geschwindigkeitsvektor des zugehörigen Rads $[\dot{x} \quad \dot{y} \quad 0]^T$, welches mit dem Betrag der Drehzahl aus (10) multipliziert wird, kann die Drehrichtung stets korrekt ermittelt werden.

3.2.5 Inverse Kinematik bei autonomer Fahrt

Zeitgleich zur autonomen Abfahrt der geplanten Trajektorie erfolgt die Rückrechnung der Lenkwinkel und Drehzahlen auf die tatsächlich zurückgelegte Trajektorie. Im Gegensatz zur Controllersteuerung, bei der das Fahrzeug lediglich translatorisch oder rotatorisch bewegt wird, werden bei der autonomen Abfahrt beide Bewegungen kombiniert. Die Rückrechnung auf die tatsächliche Trajektorie ist deshalb geringfügig komplizierter.

Die Basis der Rückrechnung liefert erneut Gl. (6). Bezogen auf Gl. (8) wird hierfür nach $\boldsymbol{v}_S$ zu $\boldsymbol{v}_S = \boldsymbol{v}_{VL} - \dot{\boldsymbol{\psi}} \times {}_S\boldsymbol{r}_{VL}$ umgestellt. Analog ergeben sich aus den übrigen drei Rädern drei weitere Formeln zur Rückrechnung auf $\boldsymbol{v}_S$.

Zur Umrechnung wurde eine Funktion entwickelt, die als Input die Drehzahlen n, die Lenkwinkel δ, die Drehgeschwindigkeit $\dot{\psi}$ sowie den Drehwinkel ψ erhält. Dieser ist notwendig, um nach Bestimmung von $\boldsymbol{v}_S$ in Fahrzeugkoordinaten eine Transformation in Weltkoordinaten durchzuführen. Die Integration der Geschwindigkeit zur Strecke erfolgt nach $\boldsymbol{s}_S = \boldsymbol{v}_S \Delta t$ in der main-Datei mit den zugehörigen Zeitstempeln. Die Abmaße des Autos sind hinterlegt, um ${}_S\boldsymbol{r}_{VL}$ zu bestimmen.

Die ursprüngliche Idee war es, ein Minimierungsproblem aufzustellen, um sowohl v_{S_x} als auch v_{S_y} zu bestimmen. Allerdings wurde diese verworfen, nachdem einer der Radnabenmotoren - vermutlich wegen defekter Hallsonden - keine zuverlässigen Drehzahlen mehr zurücklieferte.

Stattdessen wurde sich entschieden, aus den vier errechneten Größen für jeweils v_{S_x} und v_{S_y} den Median zu nutzen. Ein Ausreißer durch eine defekte Hallsonde fällt so nicht mehr ins Gewicht. Zusätzlich findet eine Mittlung zwischen Element zwei und drei statt, da der Vektor eine gerade Anzahl an Elementen besitzt.

3.3 Kalibrierung der Absolutwertgeber

Sollten die Absolutwertgeber ausgebaut werden müssen, ist es danach notwendig, sie wieder korrekt zu kalibrieren, das heißt die Spannungswerte in den Positionen -90° und + 90° auszulesen und in allen Arduinodateien ent-

sprechend in den ersten vier Zeilen wieder anzupassen. Die aktuellen Werte können in Tabelle 3 gefunden werden.

Dazu gibt es das Arduinoprogramm `oneWheelSteering.ino`. Dieses empfängt vom seriellen Monitor (`STRG + SHIFT + M` in der Arduino IDE) Zeilen nach dem Schema

```
<'wheel selection', 'nof steps'>
```

wobei `'wheel selection'` einer der vier Strings „VL“, „VR“, „HL“ und „HR“ sein kann. Mit dem entsprechenden Rad wird dann die gewählte Anzahl an Schritten gefahren. Außerdem wird der Spannungswert an allen Sensorpins jede Sekunde zurückgeschickt. Die Kommunikation funktioniert wieder analog zu Kapitel 3.1.1.

Es gibt hier keinen Sicherheitsmechanismus gegen zu große Lenkwinkel. Daher ist Vorsicht geboten, dass die Endanschläge und Kabel nicht beschädigt werden.

Für die Steuerung muss das Programm auf dem Arduino 2 gestartet werden. Wird der Arduino 1 genutzt, kann man genauso die Spannungswerte ablesen, aber das Ansteuern ist nicht möglich.

4 Endergebnisse

4.1 Xbox-Controller Steuerung und SLAM-Ergebnisse

Zur Kartierung der Umgebung, wird das Auto händisch mit einem Xbox-Controller gesteuert. Gleichzeitig werden alle 5 Sekunden Umgebungsscans mit dem Lidar aufgenommen und direkt online zur Positionsbestimmung und Kartierung mit SLAM genutzt.

Der wesentliche Ablauf des MATLAB-Programms zur Xbox-Controller Steuerung ist im Folgenden beschrieben:

1. Starten des MATLAB-Programms `matlab_xbox_remote.m`
2. Herstellung der seriellen Kommunikation zwischen MATLAB und dem Arduino 1
3. Herstellen der Verbindung zum Xbox-Controller
4. Starten des parallelen Pools
5. Herstellen der Verbindung zum Lidar-Sensor
6. Endlosschleife aus den folgenden Komponenten
   - Lesen der Xbox-Controller Eingaben und Senden der entsprechenden Befehle an den Arduino 1
   - Alle 5 Sekunden lesen des aktuellsten Scans aus dem Lidar und starten der parallelen Verarbeitung des selbigen
   - Bei eingehender Nachricht vom Arduino 1 Umrechnung der Odometriedaten in geschätzte Trajektorie
7. Plotten und speichern der Ergebnisse

Während der Fahrt führt der Arduino 1 die folgende Kommunikation durch (s. für Details Kap. 3.1.1):

- Senden der von MATLAB berechneten Lenkwinkel an den Arduino 2, der ausschließlich für die Lenksteuerung zuständig ist
- Übertragen der Soll-Drehzahlen per serieller TX/RX-Schnittstelle an einen VESC (Motorcontroller), der diese dann per CAN-Bus an die weiteren drei VESCs weiterleitet.

- Auslesen der Ist-Drehzahlen, Ist-Lenkwinkel und Ausrichtung des Fahrzeugs ψ mithilfe des Kompasses sowie Zurücksenden an MATLAB

In MATLAB werden die Ist-Daten empfangen und durch inverse Kinematik (näher erläutert in Kapitel 3.1.3) in die Ist-Position des Fahrzeugs (x, y, ψ) umgerechnet. Diese Daten können direkt geplotet werden und werden genutzt, um dem SLAM Algorithmus eine relative Pose zur Verfügung zu stellen.

Die Änderungen aus Kapitel 3.1.4, im Wesentlichen die Parallelisierung, führten zu einer flüssigen und intuitiven Steuerung und die aufgenommene Karte diente als zuverlässige Grundlage für die Autonome Fahrt, siehe die zugrundeliegende Belegungskarte in Abbildung 6. Ein weiteres Beispiel der aufgenommenen Karte mit der zurückgelegten Trajektorie ist in Abbildung 5 zu sehen. Die Karte wurde hinter dem F-Bau der THU auf dem Parkplatz aufgenommen und man erkennt sehr gut die parkenden Autos, den Eingang (Rolltor) in den F-Bau bei $(x, y) \approx (-5, -7)$, die etwas verwaschene Hecke bei $y \approx 9$ und die beiden Personen nahe dem Ursprung. **TODO: vielleicht doch nochmal fahren für bessere Karte aber auf jeden Fall in Matlab noch x und y Achse beschriften**

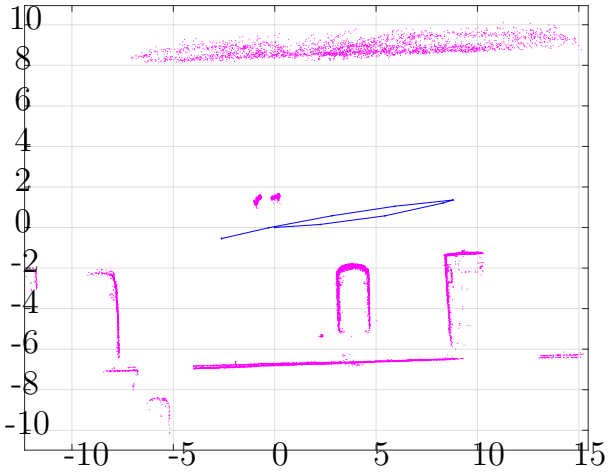


Abbildung 5: Belegungskarte mit geschätzter Trajektorie in blau

Wichtig ist, dass die Qualität des SLAM-Algorithmus stark auf Schleifenschlusserkennung beruht. Eine abgefahrene Gerade in nur eine Richtung ist deshalb Fehleranfälliger als ein Kreis, am Besten mit großem überlappenden Bereich.

4.2 Trajektorienplanung und Ausführung

Für die autonome Verfolgung einer zuvor berechneten Trajektorie wurden alle bisher entwickelten Komponenten des Fahrzeugsystems kombiniert, die hier zusammengefasst dargestellt werden.

Im Folgenden sind die wesentlichen Schritte des MATLAB-Programms zur autonomen Trajektorienabfahrt beschrieben:

1. Starten des MATLAB-Programms `matlab_autonomous.m`
2. Herstellung der seriellen Kommunikation zwischen MATLAB und dem Arduino 1
3. Festlegen der Eckdaten der Trajektorie, wie:
   - Zeitschrittweite zur Interpolation der Trajektoriendaten, die auf den Interrupt von Arduino 1 abgestimmt ist
   - Maximale Geschwindigkeit
   - Start- und Endpunkt der Trajektorie
4. Trajektorienplanung mit der Funktion `pathPlanningHybridAstar`
5. Berechnung der Lenkwinkel und Raddrehzahlen mit der Funktion `trajectoryTo8DOF`
6. Senden der Soll-Lenkwinkel und Soll-Raddrehzahlen von MATLAB an den Arduino 1 und Empfangen der Odometriedaten vom Arduino 1
7. Speichern der Ergebnisse

Während der Fahrt führt der Arduino 1 die folgende Kommunikation durch (s. für Details Kap. 3.2.1):

- Senden der von MATLAB berechneten Lenkwinkel an den Arduino 2, der ausschließlich für die Lenksteuerung zuständig ist
- Übertragen der Soll-Drehzahlen per serieller TX/RX-Schnittstelle an einen VESC (Motorcontroller), der diese dann per CAN-Bus an die weiteren drei VESCs weiterleitet
- Auslesen der Ist-Drehzahlen, Ist-Lenkwinkel und Ausrichtung des Fahrzeugs ψ mithilfe des Kompasses sowie Zurücksenden an MATLAB

In MATLAB werden die Ist-Daten empfangen und durch inverse Kinematik (näher erläutert in Kapitel 3.2.5) in die Ist-Position des Fahrzeugs (x, y, ψ) umgerechnet. Die zurückgelegte Strecke wird in derselben Grafik wie die geplante Trajektorie dargestellt, wodurch ein anschaulicher Vergleich zwischen Soll- und Ist-Werten möglich ist, vgl. Abb. 6.

Durch die Kombination der einzelnen Arbeitsschritte kann das Fahrzeug der berechneten Trajektorie folgen. Vor allem durch Störgrößen wie Bodenunebenheiten und Trägheiten des Fahrzeugs gerade zu Beginn der Fahrt mit $\boldsymbol{v} = 0$ weicht die tatsächliche Trajektorie von der geplanten Trajektorie ab.

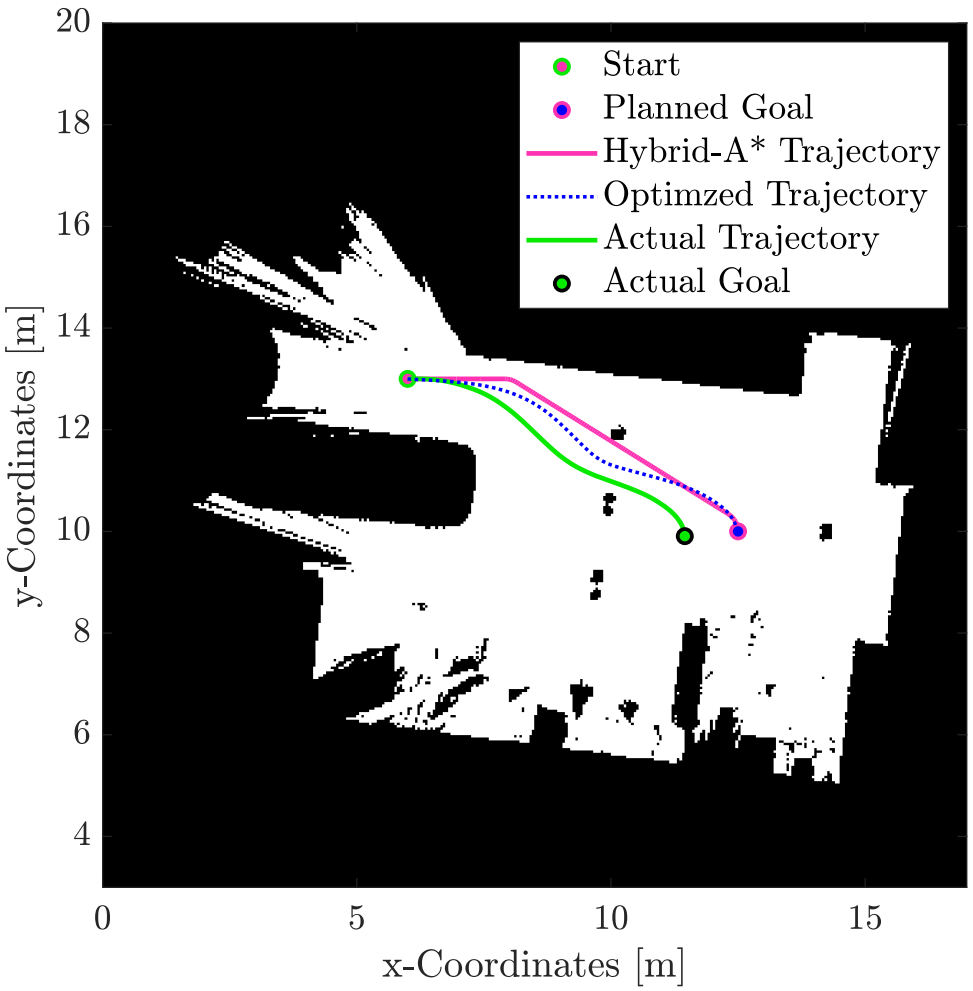


Abbildung 6: Pfadplanung mit Hybrid-A*

5 Ausblick

Ein nächster Schritt mit den gegebenen Komponenten kann daraus bestehen, die beiden Softwarekomponenten zur vereinen. Die live Lokalisierung in einer gegebenen Karte mithilfe von Lidar-Scans ermöglicht eine online Regelung der Trajektorie. Dabei wird die Soll-Position mit der Ist-Position verglichen. Vorstellbar ist ein Zurückführen des Fahrzeugs zur ursprünglichen Trajektorie oder ein Neuplanen der Trajektorie vom aktuellen Zustand hin zum ursprünglichen Endzustand.

Auch sind die Ultraschallsensoren zwar in Hardware verbaut aber noch nicht genutzt. Man kann sie verwenden, um bei einem Szenario mit beweglichen Hindernissen einen zusätzlichen Notstoppmechanismus einzubauen.

Weitere Optimierungen können auch im Bereich des SLAM-Algorithmus vorgenommen werden. Es ist potentiell möglich, den Schleifenschluss zwischen mehr als zwei Scans zu erkennen und damit die Genauigkeit zu verbessern, was aktuell noch zu lange dauert. Die Grafikkarte kann genutzt werden, um die notwendige Geschwindigkeit zu erreichen.

Außerdem sind wir mit den beiden Arduinos an die Grenzen der Echtzeitfähigkeit gestoßen. Für kompliziertere Szenarien mit z. B. den Ultraschallsensoren ist vermutlich ein leistungsfähigerer Controller, am Besten mit mehr als einem Prozessor, vorteilhaft.

Leider ist/sind ein/mehrere Endanschläge beschädigt. Um fehlerhaftes und fortlaufendes Auslösen der Endanschläge zu verhindern, müssen alle acht angebrachten Endanschläge auf Funktionstüchtigkeit überprüft werden. Eine mögliche Vorgehensweise wäre, mit einem Durchgangsprüfer alle Endanschläge zu überprüfen und jene ausfindig zu machen, die kurzzeitig das Piepen des Multimeters unterbrechen.

6 Fazit

Das Projektziel, ein autonomes Fahrzeug mit einem 8-Freiheitsgrade-Fahrwerk und umfassender Sensorik weiter zu entwickeln und zu testen, wurde erreicht. Durch die Integration verschiedener Sensoren wie Lidar, Encoder und eines Kompasses sowie die Implementierung von Pfadplanungsalgorithmen und Inverser-Kinematik-Algorithmen konnte das Fahrzeug einer geplanten Trajektorie autonom folgen.

Die initiale Steuerung über einen Xbox-Controller erwies sich als wertvoll für die Erprobung der grundlegenden Fahrzeugfunktionen und legte die Basis für die autonome Navigation. Es ergaben sich Herausforderungen bei der Ansteuerung der Radnabenmotoren und der präzisen Umsetzung der Steuerbefehle in den Schrittmotoren, wodurch gelegentlich leichte Abweichungen vom Sollwert entstanden. Durch Sensorkombination konnten die tatsächlichen Ist-Werte für Raddrehzahlen, Lenkwinkel und absoluter Ausrichtung für ein Rückwärtstracking der zurückgelegten Trajektorie genutzt werden.

Im Projektverlauf wurde zudem deutlich, wie anspruchsvoll die vollständige Vernetzung der Hardware- und Softwarekomponenten ist, insbesondere in Bezug auf die reibungslose Echtzeitkommunikation zwischen Arduino und MATLAB. Dennoch konnte ein funktionierendes System realisiert werden, das sowohl die geplante als auch die gefahrene Trajektorie aufzeichnet und vergleicht, was wertvolle Rückschlüsse auf die Steuerungsgenauigkeit erlaubt.

Insgesamt ist das Projekt ein gelungenes Beispiel für die Entwicklung und Integration einer umfassenden Fahrzeugsteuerung unter Einsatz moderner Technologien. Die daraus gewonnenen Erkenntnisse bieten wertvolle Ansätze für zukünftige Arbeiten und zeigen auf, welche Optimierungen möglich sind, um eine präzisere autonome Fahrweise zu erreichen.

Literatur

[1] Linus Capellaro. *Entwicklung eines Rahmenkonzeptes für ein innovatives Nutzfahrzeug*. Studienarbeit im Studiengang Fahrzeugtechnik and der Technischen Hochschule Ulm. 2024.

[2] Louis de Veer. *Technische Dokumentation Nimbulus-E*. Masterprojekt. Ulm, Jan. 2024.

[3] Jonathan Tasler. *Konstruktion und Aufbau eines 360°-Fahrwerks mit Steer-by-Wire Lenkung und Radnabenantrieb*. Bachelorarbeit. Technische Hochschule Ulm, 2022.

[4] *Closed-Loop-Schrittmotortreiber 0.8-7.0A 24-48VDC für NEMA 17/23/24 Schrittmotoren CL57T*. Zugriff am 3. Oktober 2024. 2024. URL: https://www.omc-stepperonline.com/de/closed-loop-schrittmotortreiber-0-8-0a-24-48vdc-fuer-nema-17-23-24-schrittmotore-cl57t?srsltid=AfmBOoofqiWvKRUoLbOlg5ytT_48eDwbShHjCDKVdZJ5RtXCfWvC6chz.

[5] Stepperonline. *User Manual CL57T(V4.0) Closed Loop Stepper Driver*. https://www.omc-stepperonline.com/de/closed-loop-schrittmotortreiber-0-8-0a-24-48vdc-fuer-nema-17-23-24-schrittmotore-cl57t. 2024.

[6] OMC-Stepperonline. *S-Serie Nema 24 Closed Loop Schrittmotor 3.0Nm 424.92oz.in Encoder 1000PPR 4000CPR*. Zugriff am 3. Oktober 2024. 2023. URL: https://www.omc-stepperonline.com/de/s-serie-nema-24-closed-loop-schrittmotor-3-0-nm-424-92oz-in-encoder-1000ppr-4000cpr-24hs34-5004d-e1000.

[7] Thomas Zimmermann. *Inbetriebnahme eines Motorreglers für einen Fahrzeugantrieb*. Studienarbeit. Bachelor of Engineering, Studiengang Fahrzeugtechnik. Ulm, Sep. 2024.

[8] Niklas Schlosser. *Erstellen von Umgebungskarten auf Basis von MATLAB SLAMAlgorithmen und 3D-LiDAR-Daten*. Bachelorarbeit an der Technischen Hochschule Ulm. 2024.

[9] Lukas Finn Braig. *Trajektorienplanung für das Nutzfahrzeug NIMBULUS-E*. Studienarbeit im Studiengang Fahrzeugtechnik and der Technischen Hochschule Ulm. 2024.

[10] *Navigation Toolbox*. MathWorks. 2024. URL: https://www.mathworks.com/products/navigation.html.

[11] *Parallel Computing Toolbox*. MathWorks. 2024. URL: https://www.mathworks.com/products/parallel-computing.html.

[12] *MATLAB Support Package for Arduino Hardware*. https://www.mathworks.com/hardware-support/arduino-matlab.html. Accessed: 2024-10-03.

[13] SolidGeek. *VescUart: A simple way to communicate with the VESC over UART*. Accessed: 2024-10-03. 2024. URL: https://github.com/SolidGeek/VescUart.

[14] Adafruit. *Adafruit BNO055 Absolute Orientation Sensor*. https://github.com/adafruit/Adafruit_BNO055. Accessed: 2024-10-03.

[15] Adafruit. *Adafruit Unified Sensor Driver*. https://github.com/adafruit/Adafruit_Sensor. Accessed: 2024-10-03.

[16] Dolubatarya. *Xbox Pairing Illustration*. Zugriff am 3. Oktober 2024. URL: https://api.dolubatarya.com/wp-content/uploads/2023/01/xbox-pairing-illustration.png.

A Anleitungen

A.1 Benötigte Toolboxen

Um das Fahrzeug mit dem Xbox-Controller zu steuern oder es rein autonom fahren zu lassen, ist es sowohl für den Arduino als auch für MATLAB notwendig, mehrere Bibliotheken zu installieren.

Die zu installierenden MATLAB Packages/Toolboxen können über den *Add-On Explorer* gefunden werden und lauten:

1. MATLAB-Navigation-Toolbox [10]
2. Parallel-Computing-Toolbox [11]
3. MATLAB Support Package for Arduino Hardware [12] (Arduino muss nach Download des Packages noch initialisiert werden.)

Je nach Voreinstellungen bei der Installation von MATLAB müssen unter Umständen noch weitere Packages/Toolboxen installiert werden.

Die zu installierenden Arduino Bibliotheken lauten:

1. TimerOne by Stoyko Dimitrov et. al., direkt über die IDE
2. VescUart [13], über Zip-Datei installieren
3. Adafruit BNO055 by Adafruit [14], direkt über die IDE
4. Adafruit Unified Sensors by Adafruit [15], direkt über die IDE

A.2 Hardware-Setup

Zur Nutzung des Autos muss mindestens ein 12 V Fahrradakku in die dafür vorgesehene Halterung geklemmt werden. Für längere Fahrt- und Testzeiträume können auch zwei Fahrradakkus gleichzeitig angeschlossen werden.

Soll das Fahrzeug per Xbox-Controller-Steuerung betrieben werden, muss zuerst der Xbox-Controller per Bluetooth mit dem PC verbunden werden. Dafür muss nach Einschalten des Controllers (Drücken des runden Xbox-Symbols in der oberen Mitte bis es leuchtet) der Pairing-Button des Controllers so lange

gedrückt werden, bis das Xbox-Symbol in hoher Frequenz weiß blinkt. Der Pairing-Button ist in Abb. 7 dargestellt.

Ist die Bluetooth-Funktion am PC aktiviert, kann er in den Bluetooth-Einstellungen über „`Gerät hinzufügen > Bluetooth`" gekoppelt werden. Die von Windows vorgeschlagenen Option, den Controller unter „`Gerät hinzufügen > Alle weiteren (z. B. Xbox-Controller)`" zu koppeln, funktioniert nicht.

Wird der Controller zwischenzeitlich mit einem anderen PC verbunden, muss bei erneutem Pairing mit dem ursprünglichen PC die alte Kopplung gelöscht und durch die zuvor beschriebene Vorgehensweise eine neue Kopplung erzeugt werden. Andernfalls wird es dem PC nicht möglich sein, eine dauerhafte Verbindung zum Controller aufzubauen.

In den beiden Folgekapiteln wird sowohl das Software-Setup zur Controllersteuerung als auch zur autonomen Fahrt näher erläutert.



Abbildung 7: Xbox Pairing Button [16]

A.3 Software-Setup Xbox-Controller Steuerung

Um das Fahrzeug mit dem Xbox-Controller steuern zu können, müssen drei verschiedene Programme auf drei verschiedenen Hardwares ausgeführt werden.

Beide Arduinos müssen per USB-Kabel mit dem steuernden PC, auf dem MATLAB samt aller nötigen Toolboxen installiert ist, verbunden sein. Die

USB-Verbindung des Arduino 2 dient lediglich der Stromversorgung. Die Datenübertragung erfolgt über serielle Kommunikationspins zwischen beiden Arduinos direkt. Da beide Arduinos mit USB verbunden sind, ist bei Datenübertragung darauf zu achten, in der Arduino IDE stets den richtigen COM-Port auszuwählen.

Der Shortcut zum Hochladen eines Programms auf den Arduino aus der Arduino IDE lautet „`CTRL + SHIFT + U`". Der Serielle Monitor wird mit „`CTRL + SHIFT + M`" geöffnet. MATLAB-Programme werden mit „`F5`" ausgeführt.

Eine Übersicht, welches Programm auf welcher Hardware auszuführen ist, findet sich in der folgenden Liste:

1. **Arduino 1:** Das Programm `Xbox Controller Steuerung\ardu1_xbox_remote\ardu1_xbox_remote.ino` muss geladen und ausgeführt werden.
2. **Arduino 2:** Das Programm `Xbox Controller Steuerung\ardu2_xbox_remote\ardu2_xbox_remote.ino` muss geladen und ausgeführt werden.
3. **PC:** Es muss in MATLAB das Programm `Xbox Controller Steuerung\matlab_xbox_remote.m` ausgeführt werden. In Zeile 55 ist der COM-Port an den Arduino 1 anzupassen. Der COM-Port variiert von PC zu PC. Es ist darauf zu achten, dass die Baudrate im MATLAB File in Zeile 55 mit der Baudrate im Setup des Arduino 1 in Zeile 105 übereinstimmt. Wir empfehlen eine Baudrate von 115200.

   Zusätzlich muss bei eventuell auftretenden Initialisierungsproblemen die `initTime` in Zeile 59 des MATLAB-Codes erhöht werden. Werte von 10 Sekunden gewährleisten im Normalfall eine sichere Initialisierung der seriellen Kommunikation zwischen MATLAB und Arduino 1. Werte von 4 Sekunden haben bei uns teilweise funktioniert.

   Sollte während der Controller-Steuerung die Umgebung mit dem Lidar aufgenommen werden sollen, muss die boolsche Variable `useLidar` in Zeile 16 auf `true` gesetzt werden. In diesem Fall erzeugt MATLAB einen Pool an parallelen Workern. Die erste Initialisierung kann hier einige Minuten dauern.

Weitere Codestellen, an denen Anpassungen sinnvoll/notwendig sind, wurden mit Kommentaren der Form `% CHANGE:` versehen.

A.4 Software-Setup autonome Fahrt

Für die autonome Fahrt, wird von dem Arudino 1 sowie von MATLAB ein anderes Programm im Vergleich zu Kap. A.3 ausgeführt. Das Arduino 2 Software-Setup bleibt unverändert, zur Vollständigkeit ist der Arduino 2 aber auch im autonomen Teil vorhanden.

1. **Arduino 1:** Das Programm `Autonome Steuerung\ardu1_autonomous\ardu1_autonomous.ino` muss geladen und ausgeführt werden.
2. **Arduino 2:** Das Programm `Autonome Steuerung\ardu2_autonomous\ardu2_autonomous.ino` muss geladen und ausgeführt werden.
3. **PC:** In MATLAB muss das Programm `Autonome Steuerung\matlab_autonomous.m` ausgeführt werden. Die in Kap. A.3 erwähnten Anpassungen des Setups müssen auch hier ggf. vorgenommen werden.

   Zusätzlich können die Variante der abzufahrenden Trajektorie in Zeile 15 - 18 und die entsprechenden Parameter in den Zeilen 38 ff. definiert werden.

   Die Kommentare mit `% CHANGE:` weißen auch hier auf sinnvolle/notwendige Änderungen hin.

A.5 Lidar nutzen

Bei der Nutzung des Lidars ist äußerste Vorsicht und Sorgfalt geboten! Es darf nicht mit den Fingern auf das Glas gefasst werden! Es sollte verhindert werden, in den Lidar zu schauen, während dieser die Umgebung scannt und Lasersignale aussendet.

Die Verkabelung darf nur nach der hier dargestellten Anleitung vorgenommen werden!

Der Lidar besitzt ein Anschlusskabel, welches bei geradeaus gerichteter Fahrt und dementsprechender Anbringung des Lidars auf der dafür vorgesehenen

Halterung an der rechten Seite des Lidargehäuses befestigt ist. Das Anschlusskabel ist sowohl für die Datenübertragung als auch für die Stromversorgung verantwortlich. Für die Datenübertragung wurde bereits ein Ethernet-Adapter an die einzelnen Datenleitungen angeschlossen. Dieser muss mit dem PC verbunden werden.

Bei der ersten Verbindung mit dem PC muss die Netzwerkschnittstelle, wie in der Arbeit von Niklas Schlosser [8] beschrieben, konfiguriert werden. Sollte es immer noch Probleme geben, kann es sein, dass man das Profil der Regel in der „Windows Defender Firewall mit erweiterter Sicherheit“ (siehe Abbildung 69 in [8]) umstellen muss.

Bei der Stromversorgung muss sorgfältig darauf geachtet werden, die Kabel **nicht** zu verwechseln. Kontraintuitiv ist das rote Kabel des Lidars der GND-Anschluss, wohingegen das blaue Kabel mit +12 versorgt werden muss. Labels befinden sich zur Hilfe an den Versorgungsleitungen des Lidars.

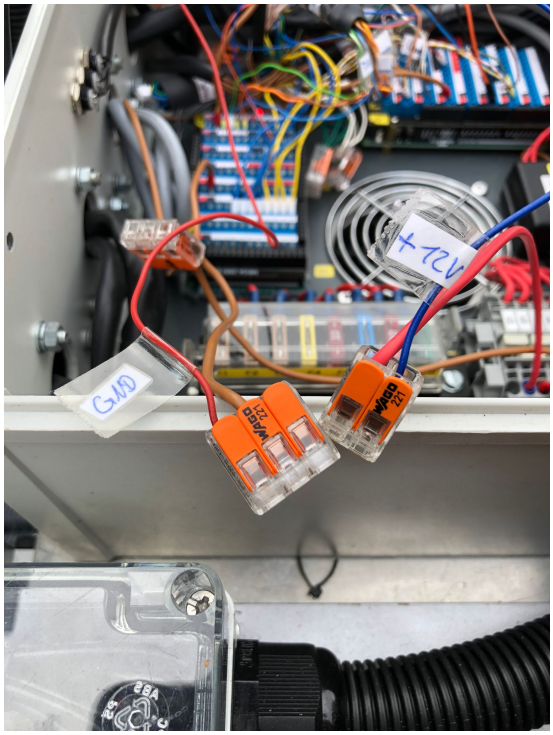
Sollte der Lidar in Betrieb sein, darf das Auto und damit der Lidar nicht wiederholt an- und ausgeschaltet werden.

Abb. 8a zeigt eine Detailansicht der Verkabelung zur Spannungsversorgung des Lidars. Abb. 8b zeigt mit größerem Bildausschnitt die Verbindung zum Lidarkabel.

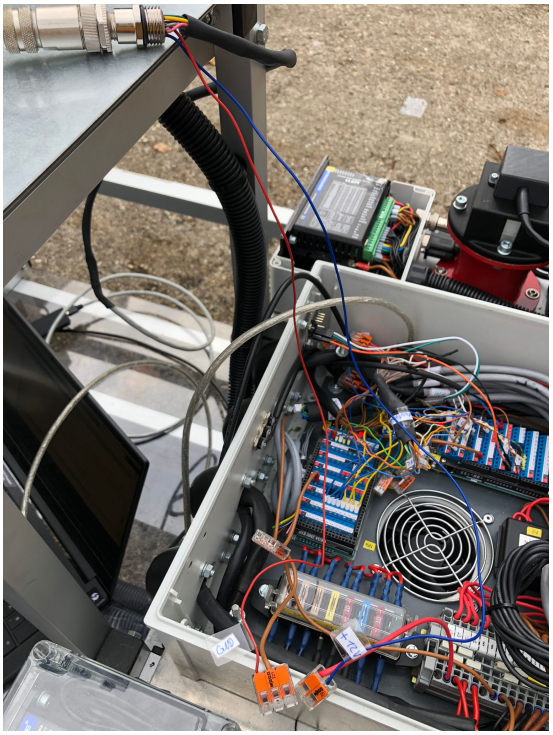
Der Lidar sitzt auch ohne zusätzliche Sicherung fest auf der Halterung und kann somit schnell angebracht sowie abgenommen werden. Sollte gewünscht sein, den Lidar zusätzlich mit einer Schraube zu sichern, muss die 3D gedruckte Halterung erneut gedruckt werden. Der Sicherungsmechanismus wurde durch eine vorhergehende Arbeit sinnfrei weggebohrt. Zur Sicherung ist eine 1/4 Zoll Fotogewindeschraube notwendig, die von unten mit einem Schlitzschraubenzieher angezogen werden kann. Die Schraube wurde bereits erworben.

A.6 One Wheel Steering

Siehe Kapitel 3.3.



(a) Detailansicht



(b) Gesamtansicht

Abbildung 8: Spannungsversorgung des Lidars

A.7 Vesc Bibliothek in Arduino

Die Bibliothek kann unter https://github.com/SolidGeek/VescUart als .zip Datei heruntergeladen und an der gewünschten Stelle auf dem PC gespeichert werden. Die Bibliothek wird dann wie unter https://docs.arduino.cc/software/ide-v1/tutorials/installing-libraries/#importing-a-zip-library beschrieben eingebunden.

A.8 Xbox-Controller Steuerung Inbetriebnahme

Die Software muss wie in Kapitel A.3 beschrieben gestartet werden. Wird der Lidar genutzt, so ist das Setup unter Kapitel A.5 beschrieben.

Nach dem Upload aller Arduino Programme und dem Start des MATLAB-Programms werden verschiedene Konfigurationen (Serielle Schnittstelle, Lidar, Controller, Paralleler Pool) durchgeführt. Dies kann einige Zeit beanspruchen. Sobald keine Ladebalken mehr zu sehen sind und die Beleuchtung der Radnabenmotoren an ist, kann das Auto gefahren werden.

Die Bedienung mit dem Xbox-Controller funktioniert wie in Kapitel 3.1.2 insbesondere Abbildung 2 beschrieben.

Nach dem Beenden des Programms mit der Taste `X` wird die Serielle Kommunikation zum Arduino 1 automatisch getrennt. Sollte das Programm anders abgebrochen werden, ein Fehler auftreten oder noch laufen, ist ein andere Kommunikation mit dem Arduino 1 nicht möglich. Insbesondere der Upload einer neuen Programmversion wird fehlschlagen. Der Befehl `delete(arduMeg)` in MATLAB löst das Problem.

Die Funkbedienung für den Notaus sollte immer bereitgehalten werden, sollte ein schnelles Agieren notwendig sein.

A.9 Autonomes Fahren

Die Software muss wie in Kapitel A.4 beschrieben gestartet werden. Es ist sicherzustellen, dass ausreichend Platz, mit großer Toleranz, für die gewählte Trajektorie zur Verfügung steht.

Nach dem Upload aller Arduino Programme und dem Start des MATLAB-Programms werden verschiedene Konfigurationen (Serielle Schnittstelle, Berechnen der Trajektorie, Berechnen der Raddrehzahlen und Lenkwinkel) durchgeführt. Dies kann einige Zeit beanspruchen. Anschließend werden zuerst die Reifen in die Lenkwinkel zum Zeitpunkt 0 gestellt und kurz darauf beginnt das Auto die autonome Fahrt.

Nachdem die Trajektorie abgefahren wurde, beendet sich das MATLAB-Programm selbständig und das Fahrzeug bleibt in der Endposition stehen.

Die Funkbedienung für den Notaus sollte immer bereitgehalten werden, sollte ein schnelles Agieren notwendig sein.

B Code-Listings

Alle Quelldateien wurden zusammen mit dieser Arbeit zur Verfügung gestellt und sind wie folgt in den Verzeichnissen zu finden

```
.
├── Autonome Steuerung
│   ├── ardu1_autonomous
│   │   ├── ardu1_autonomous.ino
│   │   └── VescNimbulus.cpp
│   ├── ardu2_autonomous
│   │   └── ardu2_autonomous.m
│   ├── buildCircleTrajectory.m
│   ├── buildRotatingAndMovingTrajectory.m
│   ├── buildRotatingTrajectory.m
│   ├── findNearestTan.m
│   ├── matlab_autonomous.m
│   ├── occMap.mat
│   ├── odometry8DOFtoRelPose.m
│   ├── pathPlanningHybridAStar.m
│   └── trajectoryTo8DOF.m
├── Reifen Kalibrierung
│   ├── oneWheelSteering
│   │   └── oneWheelSteering.ino
└── XBox Controller Steuerung
    ├── ardu1_xbox_remote
    │   ├── ardu1_xbox_remote.ino
    │   └── VescNimbulus.cpp
    ├── ardu2_xbox_remote
    │   └── ardu2_xbox_remote.ino
    ├── matlab_xbox_remote.m
    └── odometryToRelPose.m
```